



单位代码 10006

学 号 22373311

分 类 号 TP311

北京航空航天大学  
BEIHANG UNIVERSITY

## 毕业设计 (论文)

基于大语言模型与 EDA 工具反馈的  
RTL 代码自动生成与自修复研究

学 院 名 称 计算机学院

专 业 名 称 计算机科学与技术专业

学 生 姓 名 张金涛

指 导 教 师 王锐

2026 年 5 月

# 北京航空航天大学

## 本科生毕业设计（论文）任务书

### I、毕业设计（论文）题目：

基于大语言模型与 EDA 工具反馈的 RTL 代码自动生成与自修复研究

### II、毕业设计（论文）使用的原始资料（数据）及设计技术要求：

1. AutoChip 论文及开源代码

2. VerilogEval、RTLLM 评估基准

3. Icarus Verilog 开源 EDA 工具

4. 大语言模型 API（Claude、GPT 系列）

5. 北航本科毕业设计相关规范

### III、毕业设计（论文）工作内容：

1. 复现 AutoChip 风格 RTL 代码生成与反馈修复系统

2. 接入 VerilogEval 和 RTLLM 双 benchmark

3. 在三个不同能力等级的 LLM 上验证反馈循环有效性

4. 通过消融实验分离反馈信息与多采样的贡献

5. 探索反馈粒度、多轮对话和提示词策略的影响

6. 建立实验产物管理与审计机制

#### IV、主要参考资料：

- [1] Thakur S, et al. AutoChip: Automating HDL Generation Using LLM Feedback, 2023
- [2] Liu M, et al. VerilogEval: Evaluating LLMs for Verilog Code Generation, ICCAD 2023
- [3] Lu Y, et al. RTLML: An Open-Source Benchmark for RTL Generation, ASP-DAC 2024
- [4] Liu S, et al. RTLCoder: Fully Open-Source and Efficient LLM-Assisted RTL Code Generation, 2024
- [5] Tsai Y, et al. RTLFixer: Automatically Fixing RTL Syntax Errors, 2024
- [6] Yao Z, et al. HDLDebugger: Streamlining HDL Debugging, 2025
- [7] Wei J, et al. Chain-of-Thought Prompting Elicits Reasoning, NeurIPS 2022
- [8] Brown T, et al. Language Models are Few-Shot Learners, NeurIPS 2020

\_\_\_\_ 计算机 \_\_\_\_ 学院 \_\_\_\_ 计算机科学与技术 \_\_\_\_ 专业类 \_\_\_\_ 223733 \_\_\_\_ 班

学生 \_\_\_\_ 张金涛 \_\_\_\_

毕业设计(论文)时间： \_\_\_\_ 2025 \_\_\_\_ 年 \_\_\_\_ 12 \_\_\_\_ 月 \_\_\_\_ 1 \_\_\_\_ 日至 \_\_\_\_ 2026 \_\_\_\_ 年 \_\_\_\_ 5 \_\_\_\_ 月 \_\_\_\_ 30 \_\_\_\_ 日

答辩时间： \_\_\_\_ 2026 \_\_\_\_ 年 \_\_\_\_ 6 \_\_\_\_ 月 \_\_\_\_ TODO \_\_\_\_ 日

成 绩： \_\_\_\_

指导教师： \_\_\_\_

兼职教师或答疑教师（并指出所负责部分）：

\_\_\_\_\_  
\_\_\_\_\_

\_\_\_\_ 系（教研室）主任（签字）： \_\_\_\_

注：任务书应该附在已完成的毕业设计（论文）的首页。



## 本人声明

我声明，本论文及其研究工作是由本人在导师指导下独立完成的，在完成论文时所利用的一切资料均已在参考文献中列出。

作者： 张金涛

签字：

时间： 2026 年 5 月



## 基于大语言模型与 EDA 工具反馈的 RTL 代码自动生成与自修复研究

学 生：张金涛

指导教师：王锐

### 摘 要

随着集成电路设计复杂度的不断提升，寄存器传输级（RTL）代码的编写和验证已成为芯片设计流程中最耗时的环节之一。大语言模型（LLM）在代码生成领域展现出潜力，但 RTL 代码不同于普通软件代码——它描述的是并行执行的硬件结构，不仅要求语法正确，还必须通过 EDA 工具的编译和仿真验证。实际使用中，模型在零样本条件下生成的 Verilog 代码常存在语法错误、接口不匹配、时序逻辑错误和边界条件遗漏等问题，尤其在复杂设计任务上正确率有限。

针对上述问题，本文构建了一个 AutoChip 风格的 RTL 自动生成与反馈修复原型系统，实现了“生成—编译—仿真—反馈—修复”的完整迭代闭环。系统支持 VerilogEval-Human 和 RTLLM 双 benchmark 接入，通过第三方统一 API 网关实现多模型无缝切换，并支持可配置的反馈粒度（从仅编译错误到详细仿真日志）、多轮对话反馈和多种初始提示词策略。同时，系统配备了完善的实验产物管理和审计机制，确保实验结果的可追溯性和可靠性。

本文设计并执行了七组递进式实验。在 VerilogEval-Human 20 题子集上，Haiku 模型的通过率从 80%（零样本）提升至 90%（反馈）。在更具挑战性的 RTLLM\_STUDY\_12（12 道高难度设计）上，GPT-5.4 的通过率从 50% 提升至 83%，表明反馈循环在强模型和高难度任务场景下仍具有实际价值。消融实验表明，反馈信息本身贡献了约 57% 的提升，多采样机会贡献约 43%，且存在仅反馈可解而多次独立重试均无法通过的设计题目。

稳定性重复实验（每模型 4 轮）确认了核心结论的统计稳健性。反馈粒度实验揭示了倒 U 型趋势——编译反馈和简洁反馈是最稳健的选择，过于丰富的反馈可能导致信息过载。实验还发现反馈效果存在模型依赖性，不同模型对反馈信息的利用能力存在差异。



本文的研究表明，EDA 工具反馈循环对大语言模型的 RTL 自动生成具有实际价值，但反馈策略需要结合模型能力、任务难度和反馈粒度进行综合设计。

**关键词：**大语言模型；RTL 代码生成；Verilog；EDA 工具反馈；自动修复；RTLLM



# Automatic RTL Code Generation and Self-Repair Based on Large Language Models and EDA Tool Feedback

Author: Zhang Jintao

Tutor: Wang Rui

## Abstract

As integrated circuit design complexity continues to grow, writing and verifying Register Transfer Level (RTL) code has become one of the most time-consuming stages in chip design. Large Language Models (LLMs) have demonstrated potential in code generation, yet RTL code fundamentally differs from conventional software — it describes parallel hardware structures and must pass compilation and simulation verification by Electronic Design Automation (EDA) tools. In practice, Verilog code generated under zero-shot conditions frequently contains syntax errors, interface mismatches, timing logic errors, and boundary condition omissions, particularly for complex design tasks.

To address these challenges, this thesis builds an AutoChip-style prototype system for automatic RTL generation and feedback-driven repair, implementing a complete “generation–compilation–simulation–feedback–repair” iterative loop. The system supports dual benchmark integration (VerilogEval-Human and RTLLM), multi-model switching via a unified API gateway, configurable feedback granularity levels (from compile-only to rich simulation logs), multi-turn conversational feedback, and multiple prompt strategies.

Seven progressive experiment groups were designed and executed. On VerilogEval-Human (20 problems), the Haiku model’s pass rate improved from 80% (zero-shot) to 90% (feedback). On the more challenging RTLLM\_STUDY\_12 (12 high-difficulty designs), GPT-5.4’s pass rate improved from 50% to 83%, demonstrating that feedback loops remain valuable even with strong models on difficult tasks. Ablation experiments showed that error feedback information itself contributed approximately 57% of the total improvement, with multiple sampling opportunities contributing approximately 43%. Stability experiments (4 independent repetitions per



model) confirmed the statistical robustness of core conclusions. Feedback granularity experiments revealed an inverted-U trend — compile-only and succinct feedback proved most robust, while excessively rich feedback may cause information overload.

This research demonstrates that EDA tool feedback loops provide practical value for LLM-based RTL generation, but feedback strategies should be designed considering model capability, task difficulty, and feedback granularity.

Key words: Large Language Models; RTL Generation; Verilog; EDA Tool Feedback; Automatic Repair; RTLLM





## 目 录

|                                          |    |
|------------------------------------------|----|
| 1 绪论 .....                               | 7  |
| 1.1 研究背景与意义 .....                        | 7  |
| 1.2 国内外研究现状 .....                        | 7  |
| 1.2.1 LLM 辅助 RTL 代码生成 .....              | 7  |
| 1.2.2 EDA 工具反馈与自动修复 .....                | 8  |
| 1.2.3 现有工作的不足 .....                      | 8  |
| 1.3 本文研究内容 .....                         | 9  |
| 1.4 本文主要贡献 .....                         | 9  |
| 1.5 论文组织结构 .....                         | 10 |
| 2 相关技术与研究基础 .....                        | 11 |
| 2.1 Verilog 与 RTL 设计基础 .....             | 11 |
| 2.1.1 RTL 设计的基本概念 .....                  | 11 |
| 2.1.2 RTL 代码与软件代码的关键差异 .....             | 11 |
| 2.2 大语言模型代码生成基础 .....                    | 12 |
| 2.2.1 基于提示词的代码生成 .....                   | 12 |
| 2.2.2 LLM 生成 RTL 代码的常见问题 .....           | 12 |
| 2.3 EDA 编译与仿真验证流程 .....                  | 12 |
| 2.3.1 编译验证 .....                         | 12 |
| 2.3.2 功能仿真 .....                         | 13 |
| 2.3.3 编译仿真反馈的信号价值 .....                  | 13 |
| 2.4 AutoChip 方法与 EDA Feedback Loop ..... | 13 |
| 2.4.1 AutoChip 的核心思想 .....               | 13 |
| 2.4.2 反馈信息的组织 .....                      | 13 |
| 2.4.3 多候选生成与全局最优追踪 .....                 | 14 |
| 2.4.4 本文与 AutoChip 的关系 .....             | 14 |



|                                   |    |
|-----------------------------------|----|
| 2.5 RTL 生成评估基准 .....              | 14 |
| 2.5.1 VerilogEval Benchmark ..... | 14 |
| 2.5.2 RTLLM Benchmark .....       | 14 |
| 2.6 相关研究与本文定位 .....               | 15 |
| 2.6.1 相关研究方向 .....                | 15 |
| 2.6.2 本文定位 .....                  | 15 |
| 2.7 本章小结 .....                    | 15 |
| 3 系统设计与实现 .....                   | 16 |
| 3.1 系统总体架构 .....                  | 16 |
| 3.1.1 设计目标 .....                  | 16 |
| 3.1.2 模块划分 .....                  | 16 |
| 3.2 任务加载与 Benchmark 接入 .....      | 17 |
| 3.2.1 Task 数据结构 .....             | 17 |
| 3.2.2 VerilogEval-Human 加载器 ..... | 17 |
| 3.2.3 RTLLM 加载器 .....             | 17 |
| 3.2.4 适配层的必要性 .....               | 18 |
| 3.3 大语言模型调用与模型管理 .....            | 18 |
| 3.3.1 统一 API 网关架构 .....           | 18 |
| 3.3.2 重试机制与错误分类 .....             | 18 |
| 3.3.3 对实验可追溯性的意义 .....            | 18 |
| 3.4 Verilog 代码提取、编译与仿真执行 .....    | 18 |
| 3.4.1 代码提取 .....                  | 18 |
| 3.4.2 编译执行 .....                  | 18 |
| 3.4.3 仿真执行与结果解析 .....             | 19 |
| 3.4.4 评分机制 .....                  | 19 |
| 3.5 反馈循环控制机制 .....                | 19 |
| 3.5.1 单轮反馈循环 .....                | 19 |
| 3.5.2 Zero-shot 作为退化形式 .....      | 21 |



|                                      |    |
|--------------------------------------|----|
| 3.5.3 Retry-only 模式 .....            | 21 |
| 3.5.4 API 错误处理 .....                 | 21 |
| 3.6 反馈策略扩展实现 .....                   | 21 |
| 3.6.1 反馈粒度级别 .....                   | 21 |
| 3.6.2 多轮对话反馈 .....                   | 21 |
| 3.7 提示词策略扩展实现 .....                  | 22 |
| 3.7.1 四种提示策略 .....                   | 22 |
| 3.7.2 数据泄漏防护 .....                   | 22 |
| 3.7.3 策略调度机制 .....                   | 22 |
| 3.8 实验产物管理与审计机制 .....                | 22 |
| 3.8.1 实验目录结构 .....                   | 22 |
| 3.8.2 元数据与 Git 集成 .....              | 22 |
| 3.8.3 审计脚本 .....                     | 22 |
| 3.8.4 权威实验索引 .....                   | 23 |
| 3.9 本章小结 .....                       | 23 |
| 4 实验设计 .....                         | 24 |
| 4.1 实验目标与总体设计 .....                  | 24 |
| 4.1.1 实验目标 .....                     | 24 |
| 4.1.2 实验总体逻辑 .....                   | 24 |
| 4.2 Benchmark 与任务子集 .....            | 24 |
| 4.2.1 VerilogEval-Human 20 题子集 ..... | 24 |
| 4.2.2 RTLLM 基准与兼容性审计 .....           | 25 |
| 4.2.3 RTLLM_CORE_5 打通验证子集 .....      | 25 |
| 4.2.4 RTLLM_STUDY_12 正式研究子集 .....    | 25 |
| 4.3 模型设置 .....                       | 25 |
| 4.3.1 模型接入方式 .....                   | 25 |
| 4.3.2 实验模型选择 .....                   | 25 |
| 4.3.3 公共实验参数 .....                   | 27 |



|                                        |    |
|----------------------------------------|----|
| 4.4 实验条件与对照设计 .....                    | 27 |
| 4.4.1 正式实验: Zero-shot 与 Feedback ..... | 27 |
| 4.4.2 消融实验: Retry-only 条件 .....        | 27 |
| 4.4.3 稳定性重复实验 .....                    | 27 |
| 4.4.4 反馈粒度实验 .....                     | 27 |
| 4.4.5 多轮对话反馈实验 .....                   | 27 |
| 4.4.6 提示词策略实验 [8, 12] .....            | 28 |
| 4.5 评价指标与统计方法 .....                    | 28 |
| 4.5.1 主要评价指标 .....                     | 28 |
| 4.5.2 派生指标 .....                       | 28 |
| 4.5.3 统计方法 .....                       | 28 |
| 4.6 实验产物记录与可靠性控制 .....                 | 28 |
| 4.7 本章小结 .....                         | 28 |
| 5 实验结果与分析 .....                        | 29 |
| 5.1 VerilogEval-Human 基线实验 .....       | 29 |
| 5.1.1 总体结果 .....                       | 29 |
| 5.1.2 改善与失败案例 .....                    | 29 |
| 5.1.3 小结 .....                         | 29 |
| 5.2 RTLLM_STUDY_12 正式实验结果 .....        | 30 |
| 5.2.1 总体结果 .....                       | 30 |
| 5.2.2 关键发现 .....                       | 30 |
| 5.3 Feedback vs Retry-only 消融实验 .....  | 32 |
| 5.3.1 实验设计 .....                       | 32 |
| 5.3.2 GPT-5.4 消融结果 .....               | 32 |
| 5.3.3 关键证据: 仅反馈可解的题目 .....             | 32 |
| 5.3.4 Sonnet 消融结果 .....                | 34 |
| 5.4 稳定性重复实验 .....                      | 34 |
| 5.4.1 GPT-5.4 稳定性结果 .....              | 34 |



|                                   |    |
|-----------------------------------|----|
| 5.4.2 Sonnet 4.6 稳定性结果 .....      | 34 |
| 5.4.3 模型依赖性分析 .....               | 35 |
| 5.5 反馈粒度实验 .....                  | 35 |
| 5.5.1 总体结果 .....                  | 35 |
| 5.5.2 倒 U 型趋势 .....               | 35 |
| 5.5.3 编译反馈的高效性 .....              | 37 |
| 5.6 多轮对话反馈实验 .....                | 37 |
| 5.6.1 实验说明 .....                  | 37 |
| 5.6.2 GPT-5.4 结果 .....            | 37 |
| 5.6.3 Sonnet 4.6 结果 .....         | 37 |
| 5.6.4 分析 .....                    | 39 |
| 5.7 提示词策略实验 .....                 | 39 |
| 5.7.1 总体结果 .....                  | 39 |
| 5.7.2 Zero-shot 阶段的策略差异 .....     | 40 |
| 5.7.3 Feedback 的”拉平效应” .....      | 40 |
| 5.8 典型案例分析 .....                  | 40 |
| 5.8.1 float_multi: 反馈修复最难设计 ..... | 40 |
| 5.8.2 LIFObuffer: 反馈拉起弱模型 .....   | 41 |
| 5.8.3 天花板题分析 .....                | 41 |
| 5.9 综合讨论 .....                    | 41 |
| 5.9.1 反馈机制的有效性 .....              | 41 |
| 5.9.2 反馈收益的来源 .....               | 41 |
| 5.9.3 模型依赖性 .....                 | 41 |
| 5.9.4 反馈信息量的边界 .....              | 42 |
| 5.9.5 反馈组织方式与提示词策略 .....          | 42 |
| 5.9.6 局限性 .....                   | 42 |
| 5.10 本章小结 .....                   | 42 |



|                                |    |
|--------------------------------|----|
| 6 总结与展望 .....                  | 43 |
| 6.1 本文工作总结 .....               | 43 |
| 6.2 主要实验结论 .....               | 43 |
| 6.3 系统局限性 .....                | 44 |
| 6.4 后续工作展望 .....               | 44 |
| 6.4.1 扩大 benchmark 与任务规模 ..... | 44 |
| 6.4.2 更多模型与自适应反馈策略 .....       | 44 |
| 6.4.3 更精细的仿真反馈与波形分析 .....      | 44 |
| 6.4.4 检索增强与领域知识补充 .....        | 44 |
| 6.4.5 形式化验证集成 .....            | 45 |
| 6.5 本章小结 .....                 | 45 |
| 结论 .....                       | 46 |
| 致谢 .....                       | 47 |
| 参考文献 .....                     | 48 |
| 附录 A 附录 A：实验配置与运行命令 .....      | 50 |
| A.1 正式实验运行命令 .....             | 50 |
| A.2 环境配置 .....                 | 50 |
| A.3 核心代码模块索引 .....             | 50 |



## 1 绪论

### 1.1 研究背景与意义

随着集成电路设计复杂度的持续提升，寄存器传输级（RTL）代码的编写和验证已成为芯片设计流程中最耗时的环节之一。传统的 RTL 开发高度依赖工程师的专业经验，设计者需要根据功能规格手工编写 Verilog 或 VHDL 代码，并通过反复的编译、仿真和调试迭代来确保设计的功能正确性。这一过程周期长、人力成本高，且容易因人为疏忽引入难以发现的逻辑错误。

近年来，大语言模型（Large Language Model, LLM）在软件代码生成领域展现出了一定的能力，能够根据自然语言描述直接生成多种编程语言的代码。这一进展自然引发了一个研究问题：大语言模型是否也能有效地生成硬件描述语言代码，从而辅助甚至部分替代人工的 RTL 开发过程？

然而，RTL 代码生成与普通软件代码生成存在本质差异。RTL 代码描述的是并行执行的硬件结构，而非顺序执行的软件逻辑。正确的 RTL 设计不仅要求语法正确，还必须满足时序约束、位宽匹配、时钟域一致性等硬件特有要求，并最终通过 EDA 工具的编译和仿真验证。大语言模型在零样本条件下生成的 RTL 代码往往存在编译错误、接口不匹配、边界条件遗漏或算法实现偏差等问题，尤其在涉及复杂状态机、流水线设计或浮点运算等场景时，单次生成的正确率较低。

这一现实使得将大语言模型的代码生成能力与 EDA 工具的自动化验证能力相结合成为一个值得探索的方向。通过构建“生成—编译—仿真—反馈—修复”的迭代闭环，EDA 工具的编译和仿真输出可以作为结构化的反馈信号返回给模型，引导其定向修复代码中的错误。这种基于工具反馈的自动修复方法有望提升 RTL 代码的自动生成质量，降低硬件设计的人力成本。

### 1.2 国内外研究现状

#### 1.2.1 LLM 辅助 RTL 代码生成

已有研究探索了使用大语言模型从自然语言规格直接生成 Verilog 代码的可行性 [1-2]。在评估基准方面，NVlabs 发布的 VerilogEval[3] 提供了标准化的 spec-to-RTL 评估框



架, 香港科技大学发布的 RTLLM[4] 提供了涵盖更高难度设计的评估任务集。这些基准的建立推动了 RTL 生成方法的标准化评估和横向比较。

现有研究表明, 大语言模型在简单的组合逻辑和常见时序电路上具备一定的生成能力, 但在涉及复杂算法、多状态 FSM 或精确位操作的设计上, 单次生成的正确率仍有较大提升空间。

### 1.2.2 EDA 工具反馈与自动修复

AutoChip[5] 等工作提出了利用 EDA 工具反馈驱动代码修复的方法, 将传统的单次生成扩展为迭代修复过程。该类方法的核心思想是: 将编译器和仿真器输出的错误信息提取为结构化反馈, 反馈给大语言模型, 使模型能够根据明确的错误定位信息进行定向修复, 而非盲目重新生成。

此外, RTLFixer[6] 等工作针对编译错误修复进行了专门探索, HDLDebugger[7] 等工作引入了检索增强技术辅助调试。这些研究共同体现了“验证反馈驱动修复”的研究趋势。

### 1.2.3 现有工作的不足

尽管上述研究在方法探索和概念验证方面取得了有价值的进展, 但以下几个方面仍有待进一步分析:

反馈收益的来源分析不足。反馈循环带来的通过率提升中, 有多少来自错误反馈信息本身的引导, 有多少仅来自多次重试带来的采样收益? 现有工作较少对此进行严格的消融分析。

强模型与高难度场景的系统验证有限。部分研究基于较早的模型或较简单的 benchmark 进行评估。随着模型能力的快速提升, 在更强模型和更具挑战性的设计任务上, 反馈循环是否仍然有效, 需要新的实验证据。

反馈机制的影响因素缺乏系统探索。反馈信息的粒度(从仅编译错误到详细仿真日志)、反馈的组织方式(单轮独立提示 vs 多轮对话上下文)、初始提示词策略(基础提示 vs 思维链 vs 少样本示例)等维度对修复效果的影响, 尚未得到系统性的实验研究。

实验可追溯性和结论稳定性。部分复现和扩展工作在实验结果管理、数据审计和结论稳定性验证方面的机制不够完善, 影响了实验结论的可信度。





### 1.3 本文研究内容

针对上述研究现状和不足,本文首先构建了一个 AutoChip 风格的 RTL 自动生成与自修复原型系统,实现了完整的”生成—编译—仿真—反馈—修复”闭环,包括任务加载(支持 VerilogEval 和 RTLLM 双 benchmark)、大语言模型统一调用(支持多模型切换)、Verilog 代码提取与编译仿真执行、反馈信息构建与迭代控制等核心模块。

在 benchmark 接入方面,本文对 RTLLM 2.0 的 50 个设计进行了 Icarus Verilog 兼容性审计(41/50 完全可用),从中精选 12 道高难度、多类别覆盖的设计构成 RTLLM\_STUDY\_12 正式研究子集,作为后中期全部正式实验的核心任务集。

在实验研究方面,本文使用 Haiku、Sonnet 4.6 和 GPT-5.4 三个不同能力等级的模型,在 RTLLM\_STUDY\_12 上进行零样本与反馈对比实验。通过 retry-only 消融实验分离多采样与反馈信息的各自贡献;通过稳定性重复实验(每模型 4 轮)验证结论的统计稳健性;通过五级反馈粒度实验(L0-L4)研究反馈信息量的影响;通过多轮对话反馈实验比较反馈组织方式;通过四种提示词策略实验分析初始提示词工程与反馈循环的关系。

此外,本文建立了包含元数据记录、Git 版本关联、数据审计脚本和权威实验索引的完整实验管理体系,确保论文中引用的每一个实验数值均可追溯到原始数据。

### 1.4 本文主要贡献

本文的主要贡献体现在以下几个方面。在系统实现层面,本文构建了可复现的 AutoChip 风格 RTL 生成与自修复原型系统,支持双 benchmark 接入、多模型统一调用、可配置的反馈粒度和提示词策略,以及完善的实验产物管理和审计机制。

在实验验证层面,本文引入 RTLLM\_STUDY\_12 作为核心实验子集,使用三个不同能力等级的大语言模型进行交叉验证,在更难 benchmark 和更强模型条件下验证了反馈循环的有效性。通过消融和稳定性实验揭示了反馈收益的来源与模型依赖性:设计了 retry-only 消融条件将反馈循环的收益分解为多采样贡献和反馈信息贡献两个独立来源,并通过 4 轮独立重复实验验证了核心结论的统计稳定性。

在机制探索层面,本文系统研究了反馈粒度、多轮对话和提示词策略对修复效果的影响,揭示了反馈信息量的倒 U 型边界——编译反馈和简洁反馈是最稳健的选择。在工程质量层面,本文通过权威实验索引、代码审计脚本和数据审计脚本,确保了实验数据的完整性和结论的可靠性。



## 1.5 论文组织结构

本文共分六章，各章内容安排如下：

第 1 章为绪论，介绍研究背景、国内外研究现状、本文研究内容与主要贡献。

第 2 章为相关技术与研究基础，介绍 Verilog 与 RTL 设计基础、大语言模型代码生成机制、EDA 编译仿真流程、AutoChip 方法以及本文使用的 VerilogEval 和 RTLLM 评估基准。

第 3 章为系统设计与实现，详细描述本文构建的 AutoChip 风格原型系统的架构、核心模块和关键实现细节。

第 4 章为实验设计，定义本文全部实验的目标、评估基准、模型配置、实验条件、评价指标和可靠性控制方法。

第 5 章为实验结果与分析，呈现七组递进式实验的结果并进行综合讨论。

第 6 章为总结与展望，总结本文的主要工作和实验结论，讨论系统局限性，并展望后续研究方向。



## 2 相关技术与研究基础

本章介绍与本文研究密切相关的技术基础和评估基准，包括 Verilog 与 RTL 设计基础、大语言模型代码生成机制、EDA 编译仿真流程、AutoChip 方法与反馈循环思想，以及本文使用的两个评估基准。本章的目的是为第 3 章的系统设计和第 4–5 章的实验分析建立必要的技术背景。

### 2.1 Verilog 与 RTL 设计基础

#### 2.1.1 RTL 设计的基本概念

寄存器传输级 (Register Transfer Level, RTL) 是数字集成电路设计中最重要抽象层次之一。在 RTL 层面，设计者使用硬件描述语言描述数据在寄存器之间的传输和变换逻辑，这些描述随后通过综合工具转化为门级网表，最终映射到物理芯片。

Verilog 是工业界最广泛使用的硬件描述语言之一。一个 Verilog 设计的基本单元是模块 (module)，每个模块定义一组输入和输出端口，并在内部实现相应的硬件功能。模块内部的逻辑可分为两大类：组合逻辑（输出仅取决于当前输入，如加法器、多路选择器）和时序逻辑（输出还取决于过去的状态，如计数器、状态机）。有限状态机 (FSM) 是时序逻辑的典型代表，广泛应用于协议控制、序列检测等场景。

#### 2.1.2 RTL 代码与软件代码的关键差异

RTL 代码的自动生成比普通软件代码生成更具挑战性，主要源于以下本质差异。在并行性方面，软件代码通常按顺序执行，而 Verilog 中的 always 块和 assign 语句在仿真语义下是并行执行的，生成代码时需要正确处理多个并发信号的时序关系。在时钟与复位方面，时序逻辑依赖时钟边沿触发和复位信号初始化，遗漏复位逻辑或错误的时钟边沿敏感列表会导致仿真行为与预期不一致。在硬件结构约束方面，每一行 Verilog 代码最终对应物理硬件，不当的描述可能导致综合后的电路行为异常。在位宽语义方面，信号的位宽需要精确匹配，符号扩展、截断、拼接等位操作的错误是 RTL 代码中常见的 bug 来源。

这些差异意味着，即使大语言模型在软件代码生成上表现良好，其生成的 RTL 代码仍可能包含硬件语义层面的错误，需要 EDA 工具进行验证。



## 2.2 大语言模型代码生成基础

### 2.2.1 基于提示词的代码生成

大语言模型 (Large Language Model, LLM) 通过在大规模文本和代码语料上的预训练, 获得了根据自然语言描述生成代码的能力 [8]。在代码生成任务中, 用户提供一段包含需求描述的提示词 (prompt), 模型基于其内部的概率分布逐 token 采样生成代码。

采样温度 (temperature) 是控制生成随机性的关键参数。较低的温度使模型倾向于选择概率最高的 token, 生成更确定性的输出; 较高的温度增加了输出的多样性, 有助于探索不同的解题路径。在本文的实验中, 温度统一设置为 0.7, 以平衡生成质量与多样性。

### 2.2.2 LLM 生成 RTL 代码的常见问题

尽管大语言模型具备一定的 Verilog 代码生成能力, 但在实际使用中常出现以下问题: 语法错误 (模块声明不完整、端口列表与描述不匹配、信号未声明等) 导致 EDA 编译失败; 接口不匹配 (生成的模块名称或端口定义与测试平台期望不一致) 导致编译时的模块实例化失败; 边界条件错误 (代码在大多数测试用例上正确但在特定边界条件上出错); 以及逻辑理解偏差 (模型对复杂算法的理解不够精确导致算法层面的实现错误)。

这些问题使得零样本生成通常难以覆盖复杂的 RTL 设计任务。这一现实为引入编译和仿真反馈来迭代修复代码提供了动机。

## 2.3 EDA 编译与仿真验证流程

### 2.3.1 编译验证

硬件设计的编译阶段将 Verilog 源代码解析为内部表示, 并检查语法正确性、信号声明一致性和模块接口匹配性。编译器输出的错误信息可以精确定位代码中的结构性问题。

本文使用 Icarus Verilog[9] (iverilog) 作为编译工具。iverilog 是一个开源的 Verilog 编译器和仿真器, 支持 IEEE 1364-2005 标准和部分 SystemVerilog 2012 特性 (通过 -g2012 选项启用)。选择开源工具的原因包括: 可免费获取, 适合本科毕设环境; 易于集成到自动化流程中; 实验结果完全可复现, 不依赖商业许可。



### 2.3.2 功能仿真

编译成功后, 使用 VVP 仿真引擎执行编译产生的仿真程序。仿真过程中, 测试平台 (testbench) 对待测设计施加激励信号, 并将设计输出与预期结果进行比较。

仿真结果通常以通过/失败的形式呈现。VerilogEval 的测试平台通过逐样本比较待测模块与参考模块的输出, 报告不匹配的样本数; RTLLM 的测试平台则直接输出 “Your Design Passed” 或失败统计。这些结构化的仿真输出为后续的反馈提供了可解析的信号。

### 2.3.3 编译仿真反馈的信号价值

编译和仿真的输出信息具有重要的反馈价值: 编译错误可以精确指出语法或接口层面的问题, 仿真失败则揭示逻辑层面的功能错误。将这些信息反馈给大语言模型, 使模型能够定向修复代码, 而非盲目重新生成。这一思想构成了 AutoChip 方法的核心。

## 2.4 AutoChip 方法与 EDA Feedback Loop

### 2.4.1 AutoChip 的核心思想

AutoChip[5] 提出了一种基于 EDA 工具反馈的 Verilog 代码迭代修复方法。其核心闭环为: 大语言模型根据自然语言描述生成 Verilog 代码, EDA 工具对代码进行编译和仿真验证, 验证结果中的错误信息被提取并组织为结构化反馈, 反馈与原始任务描述一起构成新的提示词, 模型据此生成修复后的代码。此过程反复迭代, 直到代码通过全部测试或达到最大迭代次数。

与传统的零样本生成相比, 这种 “生成—验证—反馈—修复” 闭环具有两方面优势: 一是能够利用 EDA 工具提供的精确错误定位信息; 二是通过多轮迭代逐步逼近正确解, 尤其适用于模型首次生成 “接近正确” 但存在局部错误的场景。

### 2.4.2 反馈信息的组织

反馈信息的组织方式对修复效果有影响。简洁反馈 (succinct feedback) 的基本思想是: 不直接将完整的编译日志或仿真波形传递给模型, 而是提取关键信息 (如错误行号、不匹配样本数、仿真输出摘要), 在控制 token 消耗的同时保留有效的修复线索。



### 2.4.3 多候选生成与全局最优追踪

在每轮迭代中，系统可生成多个候选代码（k-candidate generation），并从中选择最优候选。全局最优追踪机制确保反馈始终基于历史最佳代码的错误信息构建，避免因当前轮代码质量暂时回退而丢失历史最优解。

### 2.4.4 本文与 AutoChip 的关系

本文复现了 AutoChip 的核心闭环思想，并在以下维度进行了扩展：在 VerilogEval 和 RTLLM 两个 benchmark 上验证有效性；使用多个不同能力等级的大语言模型（Haiku、Sonnet、GPT-5.4）进行交叉验证；设计了 retry-only 消融条件以分离多采样与反馈信息的各自贡献。

此外，本文还通过稳定性重复实验验证结论的统计稳健性，并探索了反馈粒度、多轮对话反馈和初始提示词策略等多个优化方向。

## 2.5 RTL 生成评估基准

### 2.5.1 VerilogEval Benchmark

VerilogEval[3] 是 NVlabs 发布的 Verilog 代码生成评估基准，发表于 ICCAD 2023。其人工编写子集（VerilogEval-Human）包含 156 道来自 HDLBits[10] 的硬件设计任务，覆盖组合逻辑、时序逻辑、有限状态机等多种类型，难度从基础的线连接到复杂的 Conway 生命游戏。

VerilogEval 的评估方式为 spec-to-RTL：给定自然语言描述和模块接口定义，要求生成完整的模块实现。本文在中期阶段选取 VerilogEval-Human 的 20 道题目作为基线实验任务集。

### 2.5.2 RTLLM Benchmark

RTLLM[4] 是香港科技大学发布的 RTL 设计生成评估基准，发表于 ASP-DAC 2024。该基准包含 50 个涵盖算术运算、控制逻辑、存储器和杂项功能的硬件设计任务，难度从基础加法器到 IEEE 754 浮点乘法器、流水线乘法器和交通灯控制器等复杂设计。

与 VerilogEval 相比，RTLLM 的任务普遍更为复杂。本文在接入 RTLLM 前进行了全量 Icarus Verilog 兼容性审计：50 个设计中 46 个编译通过（92%），41 个仿真通过





(82%)。基于审计结果，本文从 41 个完全可用的设计中选取了 RTLLM\_STUDY\_12 正式研究子集。

## 2.6 相关研究与本文定位

### 2.6.1 相关研究方向

围绕大语言模型与硬件设计的交叉领域，已有研究主要覆盖以下方向：LLM 直接生成 RTL[1-2, 11]，RTL 生成评估方法 [3-4]，EDA 反馈与自修复 [5-7]，提示词工程 [8, 12]，以及多轮交互与代理方法 [13-14]。

### 2.6.2 本文定位

本文的定位是：构建一个可复现的 AutoChip 风格闭环系统，并在更强模型、更难 benchmark 上系统分析反馈循环的有效性、边界和影响因素。本文不以训练新模型或开发新的商业 EDA 工具为目标，而是聚焦于实验性贡献。

## 2.7 本章小结

本章介绍了本文研究所涉及的技术基础：Verilog 与 RTL 设计的基本概念及其与软件代码的差异，大语言模型生成代码的机制及其在 RTL 任务上的局限，EDA 编译与仿真验证的流程及其反馈信号价值，以及 AutoChip 方法的核心思想。同时介绍了本文使用的两个评估基准——VerilogEval-Human 和 RTLLM——及其在实验中的定位。这些技术基础为第 3 章的系统设计与实现奠定了背景。



### 3 系统设计与实现

本章介绍本文所构建的 AutoChip 风格 RTL 自动生成与反馈修复原型系统的总体架构、核心模块设计与关键实现细节。该系统以”生成—编译—仿真—反馈—修复”为核心闭环，支持多种 benchmark 接入、多模型切换、多种反馈策略与提示词策略，为后续实验提供了统一的可复现实验平台。

#### 3.1 系统总体架构

##### 3.1.1 设计目标

本系统的核心目标是构建一个可扩展、可审计的 AutoChip 风格实验平台 [5]，具体包括：接入多种公开 RTL benchmark，将不同格式的題目统一为内部 Task 表示；调用大语言模型生成 Verilog 代码，并通过 EDA 工具链进行编译与仿真验证；将验证结果以结构化反馈形式返回模型，驱动迭代修复；记录完整实验过程元数据，确保结果可追溯、可审计。

##### 3.1.2 模块划分

系统由以下核心模块组成：任务加载器（Task Loader）负责从不同 benchmark 目录中读取题目描述、模块接口与测试平台文件，统一转化为内部 Task 数据结构；提示词构建器（Prompt Builder）根据任务描述、模块接口及可选的反馈信息，构建发送给大语言模型的提示词；大语言模型客户端（LLM Client）封装 API 调用逻辑，通过第三方统一网关接入多种模型；Verilog 提取器从模型返回的文本中提取代码块；Verilog 执行器调用 Icarus Verilog 编译器和 VVP 仿真器；评分器基于编译与仿真结果对候选代码打分；反馈循环控制器实现完整的迭代闭环；实验产物管理器负责元数据记录与结果序列化。

图3.1展示了系统的整体数据流：任务加载器从 benchmark 目录读取题目，提示词构建器将题目描述与反馈信息组织为提示词，LLM 客户端调用模型获得自然语言回复，Verilog 提取器从中提取代码，执行器进行编译与仿真，评分器给出分数，反馈循环控制器根据分数决定是否继续迭代。



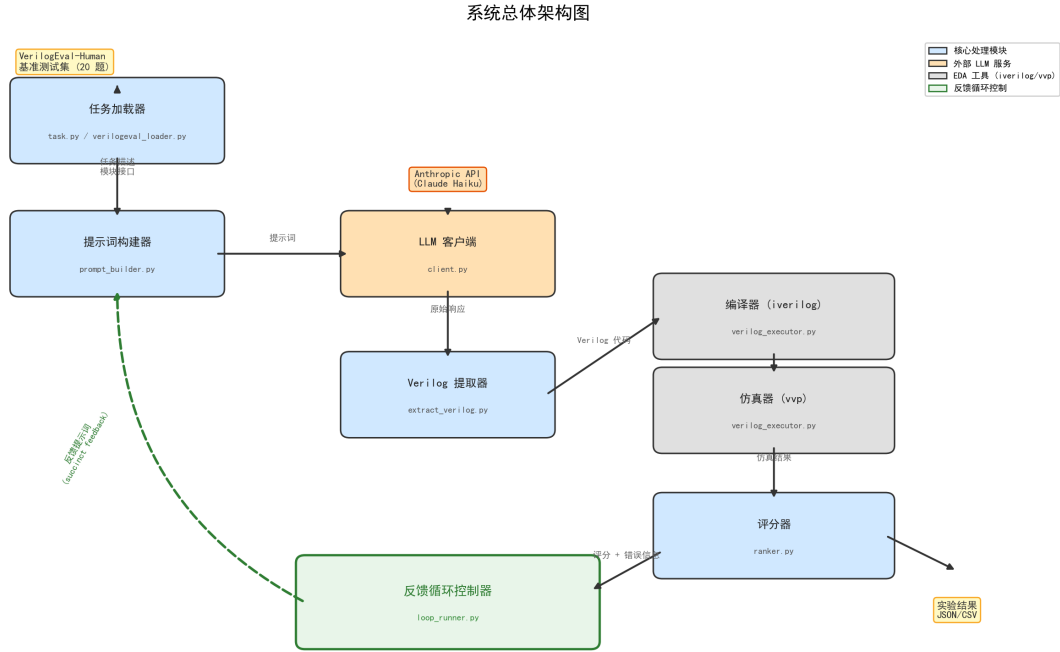


图 3.1 AutoChip 风格 RTL 生成与反馈修复系统架构

## 3.2 任务加载与 Benchmark 接入

### 3.2.1 Task 数据结构

为了屏蔽不同 benchmark 在文件组织和题目格式上的差异，系统定义了统一的 Task 数据结构，包含四个核心字段：任务名称（name）、自然语言描述（description）、模块接口声明（module\_header）以及测试平台文件路径（testbench\_path）。

### 3.2.2 VerilogEval-Human 加载器

VerilogEval[3] 采用 spec-to-RTL 模式，提供自然语言描述和 TopModule 接口定义。VerilogEval 加载器的核心逻辑是：读取自然语言描述作为 description，读取接口声明作为 module\_header，将参考实现与测试平台合并后写入临时文件作为 testbench\_path。

### 3.2.3 RTLLM 加载器

RTLLM[4] 的每个题目目录包含一个自然语言设计描述文件和一个独立的测试平台文件。RTLLM 加载器通过递归遍历分类目录来发现所有题目，对每个包含 design\_description.txt 和 testbench.v 的目录创建一个 RTLLMProblem 对象。



### 3.2.4 适配层的必要性

两个 benchmark 在文件结构、描述格式、测试平台组织和仿真输出格式上的差异，使得专用加载器成为必要。统一的 Task 接口确保了反馈循环控制器和评分器等下游模块无需关心数据来源。

## 3.3 大语言模型调用与模型管理

### 3.3.1 统一 API 网关架构

本系统通过第三方统一 API 网关接入多种大语言模型。网关的核心价值在于：保持 API 密钥和服务端点固定不变，仅通过模型名称参数切换不同的底层模型。

### 3.3.2 重试机制与错误分类

系统内置了指数退避重试机制：对可重试的瞬态错误自动重试最多 3 次，退避间隔为 2s、5s、10s。当所有重试均失败时，错误信息被封装为结构化的 APIError 对象，记录到该候选结果中。

### 3.3.3 对实验可追溯性的意义

上述设计确保了两个关键属性：任何实验结果都可以精确追溯到使用的模型名称和调用参数；API 错误不会污染实验数据。

## 3.4 Verilog 代码提取、编译与仿真执行

### 3.4.1 代码提取

Verilog 提取器首先移除 Markdown 代码围栏标记，然后使用正则表达式匹配所有 module ... endmodule 结构。该设计对 CoT 提示策略具有兼容性。

### 3.4.2 编译执行

编译阶段使用 Icarus Verilog[9] 将生成的设计代码与测试平台一起编译，启用-g2012 选项支持 SystemVerilog 2012 标准，并设置 30 秒超时保护。



### 3.4.3 仿真执行与结果解析

仿真阶段使用 VVP 执行编译产生的仿真二进制文件，设置 60 秒超时保护。解析器实现了双格式支持：VerilogEval 格式匹配 mismatched samples is N out of M 模式；RTLMM 格式匹配 Your Design Passed 或 Test completed with N/M failures。

### 3.4.4 评分机制

评分器基于 AutoChip 论文定义的排序规则，将编译与仿真结果映射为一个标量分数，如表3.1所示。

表 3.1 评分机制定义

| 情况             | 分数      | 含义              |
|----------------|---------|-----------------|
| 未提取到有效 Verilog | -2.0    | 代码提取失败          |
| 编译失败           | -1.0    | 存在语法或结构错误       |
| 编译有警告但无仿真结果    | -0.5    | 可能存在问题          |
| 编译成功但无仿真数据     | 0.0     | 无法评估正确性         |
| 仿真部分通过         | 0 ~ 1.0 | (总样本 - 不匹配)/总样本 |
| 仿真完全通过         | 1.0     | 设计正确            |

## 3.5 反馈循环控制机制

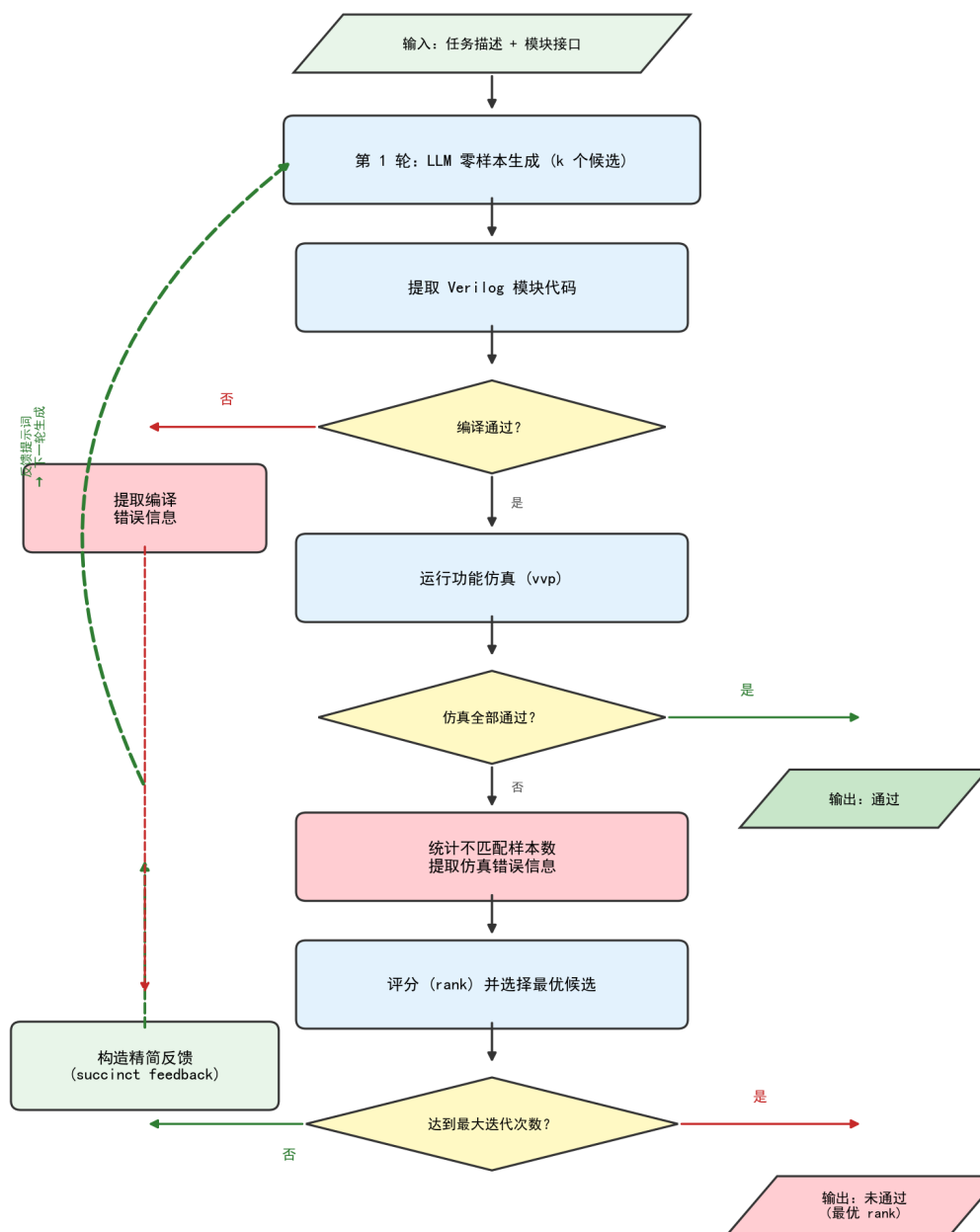
反馈循环是本系统的核心，实现了 AutoChip 论文提出的”生成—验证—反馈—修复”迭代闭环。

### 3.5.1 单轮反馈循环

单轮反馈循环（Single-turn Feedback Loop）的执行流程如下：第 1 轮使用初始提示词生成  $k$  个候选代码，分别编译、仿真、评分，选出全局最优候选；第 2 轮及后续，将全局最优候选的代码和对应的编译/仿真错误信息组织为反馈提示词，再次生成  $k$  个候选。循环直到某候选通过（rank=1.0）或达到最大迭代次数（默认 5 轮）。

全局最优追踪机制确保了反馈循环具有单调性。

AutoChip 反馈循环逻辑流程图



注：每轮生成 k 个候选（本实验 k=3），选 rank 最高者作为下一轮的基础

图 3.2 反馈循环控制流程



### 3.5.2 Zero-shot 作为退化形式

零样本模式可视为反馈循环的退化形式： $k = 1$ 、 $\text{max\_iterations}=1$ 。这一设计使得零样本基线实验和反馈实验共享相同的代码路径，确保了实验条件的公平性。

### 3.5.3 Retry-only 模式

为了区分反馈循环中“多次重试”和“错误反馈信息”各自的贡献，系统实现了 **retry-only** 模式：在该模式下，所有迭代均使用初始提示词，不包含任何错误反馈信息。

### 3.5.4 API 错误处理

当某个候选的 API 调用失败时，该候选被标记为 `api_error` 但不终止整个迭代。只有当一轮迭代中所有  $k$  个候选全部因 API 错误而失败时，循环才提前终止。

## 3.6 反馈策略扩展实现

### 3.6.1 反馈粒度级别

为了研究反馈信息量对修复效果的影响，系统实现了三种反馈粒度级别：Level 2（Compile-only，仅编译反馈）仅反馈编译阶段的错误信息；Level 3（Succinct，简洁反馈）在编译反馈的基础上增加仿真失败时的不匹配样本数和仿真输出前 40 行，是系统的默认级别；Level 4（Rich，丰富反馈）进一步扩展仿真输出到前 80 行并添加显式分析提示。

结合 `loop_runner` 中零样本（Level 0）和 **retry-only**（Level 1），系统共支持五个梯度的反馈水平（L0-L4）。

### 3.6.2 多轮对话反馈

除单轮反馈外，系统还实现了多轮对话反馈模式（Multi-turn Feedback）。与单轮模式中每次 API 调用均为独立请求不同，多轮模式维护一个持续增长的对话历史。

多轮对话的关键设计考虑包括：初始消息使用简化版提示词；反馈消息不包含上一轮的代码（因为代码已存在于对话历史中）；反馈信息必须描述模型在对话中最后一轮生成的代码的错误，确保反馈与上下文一致。



### 3.7 提示词策略扩展实现

#### 3.7.1 四种提示策略

为了研究初始提示词工程对 RTL 生成质量的影响，系统实现了四种可切换的提示策略 [8, 12]: P0 (Base Prompt, 直接指令式提示)、P1 (CoT Prompt, 思维链提示)、P2 (Few-shot Prompt, 少样本提示) 和 P3 (Few-shot + CoT Prompt)。

#### 3.7.2 数据泄漏防护

Few-shot 示例的选择需要避免数据泄漏。系统使用的两个示例——8 位加法器和 4 位计数器——均不在正式实验使用的 STUDY\_12 子集中。

#### 3.7.3 策略调度机制

提示策略通过 PromptStrategy 枚举类型定义，由实验脚本通过命令行参数传入。策略选择被记录在实验元数据中，确保结果可追溯。

### 3.8 实验产物管理与审计机制

#### 3.8.1 实验目录结构

系统采用类型化的实验产物目录结构。每个实验运行拥有唯一的 run\_id，其目录下包含 summary.json、details.json 和 summary.csv 三类产物文件。

#### 3.8.2 元数据与 Git 集成

每次实验运行的元数据自动采集当前 Git 仓库状态，包括 commit hash、分支名称和工作区是否有未提交的修改。

#### 3.8.3 审计脚本

代码审计脚本 (audit\_project.py) 执行 24 项自动化检查；数据审计脚本 (audit\_data.py) 遍历所有实验运行目录，从原始 summary.json 中重新计算通过率，与文档记录的结论交叉验证。



#### 3.8.4 权威实验索引

为防止论文撰写过程中误引用已废弃的实验结果，系统建立了权威实验索引文档，明确标注每个实验阶段的状态。

### 3.9 本章小结

本章详细介绍了 AutoChip 风格 RTL 自动生成与反馈修复原型系统的设计与实现。系统以统一的 Task 接口屏蔽了不同 benchmark 的格式差异，通过第三方统一 API 网关实现了多模型无缝切换，通过可配置的反馈粒度和提示词策略支持了多维度的实验对比。该系统为第 4 章的实验设计和第 5 章的实验分析提供了坚实的工程基础。



## 4 实验设计

本章定义本文全部实验的目标、评估基准、模型配置、实验条件、评价指标与可靠性控制方法。所有实验设置均来自真实的实验脚本与权威文档，具体实验结果将在第 5 章中呈现和分析。

### 4.1 实验目标与总体设计

#### 4.1.1 实验目标

本文的实验旨在系统性地回答以下研究问题：（1）AutoChip 风格的 EDA 反馈循环是否能稳定提升大语言模型生成 Verilog 代码的正确性？（2）反馈循环带来的通过率提升中，有多少来自错误反馈信息本身，有多少仅来自多次重试机会？（3）上述结论在多次独立重复实验下是否稳定？（4）不同粒度的反馈信息对修复效果有何影响？（5）多轮对话式反馈是否优于单轮反馈？（6）初始提示词的工程优化对生成质量有多大影响？（7）上述结论在不同能力水平的模型上是否一致？

#### 4.1.2 实验总体逻辑

围绕上述研究问题，本文设计了七组递进式实验。首先在 VerilogEval-Human 基准上完成中期基线实验；其次在 RTLLM\_STUDY\_12 上进行正式实验矩阵；然后通过消融实验和稳定性重复实验回答因果归因和统计置信度问题；最后通过反馈粒度、多轮对话和提示词策略实验探索优化方向。

### 4.2 Benchmark 与任务子集

#### 4.2.1 VerilogEval-Human 20 题子集

VerilogEval-Human[3] 是 NVlabs 发布的 Verilog 代码生成评估基准的人工编写子集。本文在中期阶段选取其中 20 个题目作为基线实验的任务集，覆盖组合逻辑、时序逻辑、有限状态机等多种类型。





#### 4.2.2 RTLLM 基准与兼容性审计

RTLLM 2.0[4] 包含 50 个硬件设计任务。由于本文使用 Icarus Verilog 作为编译仿真工具，在接入前进行了全量兼容性审计，结果如表4.1所示。

表 4.1 RTLLM 2.0 Icarus Verilog 兼容性审计结果

| 指标        | 数量 | 比例   |
|-----------|----|------|
| 总设计数      | 50 | 100% |
| 编译通过      | 46 | 92%  |
| 仿真通过      | 41 | 82%  |
| 编译失败      | 4  | 8%   |
| 编译通过但仿真异常 | 5  | 10%  |

#### 4.2.3 RTLLM\_CORE\_5 打通验证子集

在进入正式实验前，本文先定义了一个 5 题的最小验证子集 RTLLM\_CORE\_5，用于端到端验证。CORE\_5 的结果不作为论文正式结论的数据来源，在权威实验索引中被标记为 Exploratory。

#### 4.2.4 RTLLM\_STUDY\_12 正式研究子集

RTLLM\_STUDY\_12 是本文全部正式实验的核心任务集，包含 12 个精心选取的题目，如表4.2所示。

### 4.3 模型设置

#### 4.3.1 模型接入方式

本系统通过第三方统一 API 网关接入多种大语言模型，模型名称通过 LLM\_MODEL 环境变量或-model 命令行参数指定。

#### 4.3.2 实验模型选择

本文的实验涉及三个主要模型，代表三个不同的能力等级，如表4.3所示。



表 4.2 RTLLM\_STUDY\_12 任务组成与类别分布

| #  | 设计名               | 类别                         | 难度    |
|----|-------------------|----------------------------|-------|
| 1  | float_multi       | Arithmetic/Other           | ★★★★★ |
| 2  | multi_booth_8bit  | Arithmetic/Multiplier      | ★★★★  |
| 3  | multi_pipe_8bit   | Arithmetic/Multiplier      | ★★★★  |
| 4  | div_16bit         | Arithmetic/Divider         | ★★★★  |
| 5  | adder_bcd         | Arithmetic/Adder           | ★★★   |
| 6  | fsm               | Control/FSM                | ★★★   |
| 7  | sequence_detector | Control/FSM                | ★★★   |
| 8  | JC_counter        | Control/Counter            | ★★★   |
| 9  | LIFObuffer        | Memory/LIFO                | ★★★   |
| 10 | LFSR              | Memory/Shifter             | ★★★   |
| 11 | traffic_light     | Miscellaneous/Others       | ★★★★  |
| 12 | freq_divbyfrac    | Miscellaneous/Freq divider | ★★★★  |

表 4.3 实验模型与角色说明

| 模型名称              | 提供商       | 角色   | 实验覆盖                         |
|-------------------|-----------|------|------------------------------|
| claude-haiku-4-5  | Anthropic | 较弱基线 | VerilogEval 基线 + RTLLM 正式    |
| claude-sonnet-4-6 | Anthropic | 中强模型 | 正式 + 消融 + 稳定性 + 粒度 + 多轮      |
| gpt-5.4           | OpenAI    | 强模型  | 正式 + 消融 + 稳定性 + 粒度 + 多轮 + 策略 |



### 4.3.3 公共实验参数

除另有说明外, 本文所有实验使用以下统一参数:  $\text{temperature}=0.7$ ,  $\text{max\_tokens}=4096$ ,  $k=3$  (每轮候选数),  $\text{max\_iterations}=5$ , 默认反馈级别为 L3 Succinct, 默认提示策略为 P0 Base。

## 4.4 实验条件与对照设计

### 4.4.1 正式实验: Zero-shot 与 Feedback

正式实验矩阵在 RTLLM\_STUDY\_12 上对三个模型分别运行两种条件: Zero-shot ( $k=1$ , 不迭代, 无反馈) 和 Feedback ( $k=3$ , 最多 5 轮, L3 Succinct 反馈) [5]。

### 4.4.2 消融实验: Retry-only 条件

设计了三条件消融实验: Zero-shot (预算 1)、Retry-only (预算最多 15, 无反馈) 和 Feedback (预算最多 15, L3 反馈)。

### 4.4.3 稳定性重复实验

对消融实验进行了独立重复: GPT-5.4 和 Sonnet 4.6 各 4 轮, 每轮完全独立的 API 调用。

### 4.4.4 反馈粒度实验

设计了五级反馈粒度实验: L0 Zero-shot、L1 Retry-only、L2 Compile-only、L3 Succinct、L4 Rich。

### 4.4.5 多轮对话反馈实验

设计了四条件对照实验, 在 STUDY\_12 的 6 题代表性子集 (Multiturn-6) 上运行。需要说明的是, 本实验的初始版本 (v1) 存在代码 bug, 已修正为 v2 版本。本文报告的所有多轮对话结果均来自 v2。



#### 4.4.6 提示词策略实验 [8, 12]

设计了四种提示策略（P0 Base、P1 CoT、P2 Few-shot、P3 Few-shot+CoT）的对比实验，每种策略均运行 Zero-shot 和 Feedback 两种条件。该实验使用 GPT-5.4 模型。

### 4.5 评价指标与统计方法

#### 4.5.1 主要评价指标

通过率（Pass Rate）是本文的首要评价指标，定义为在给定条件下通过测试平台全部测试用例的任务数占总任务数的比例。Rank 分数作为辅助指标，取值范围为  $[-2.0, 1.0]$ 。

#### 4.5.2 派生指标

Feedback 改善题数和 API 错误率分别衡量反馈循环的净贡献和实验数据的完整性。

#### 4.5.3 统计方法

对于稳定性重复实验，本文报告 4 轮重复的均值和标准差。对于单次运行的实验，在结论表述中注明其局限性。

### 4.6 实验产物记录与可靠性控制

审计与验证机制包括代码审计（24 项检查）和数据审计（覆盖全部 7 个实验类别），两项均已通过。权威实验索引明确标注每个实验阶段的状态。

### 4.7 本章小结

本章系统地定义了本文全部实验的设计方案。实验以 VerilogEval-Human 和 RTLLM\_STUDY\_12 为评估基准，使用三个不同能力等级的大语言模型，通过七组递进式实验回答关于反馈机制有效性、价值归因、信息量影响、组织方式和提示词策略的研究问题。这些设计为第 5 章的实验结果分析奠定了方法论基础。



## 5 实验结果与分析

本章呈现全部七组实验的结果，并进行系统性分析。所有数据均以权威实验索引为准。

### 5.1 VerilogEval-Human 基线实验

本节报告中期阶段在 VerilogEval-Human 20 题子集上使用 Haiku 模型的基线实验结果。

#### 5.1.1 总体结果

表 5.1 VerilogEval-Human 20 题基线实验结果

| 条件        | 通过数   | 通过率      |
|-----------|-------|----------|
| Zero-shot | 16/20 | 80%      |
| Feedback  | 18/20 | 90%      |
| 提升        | +2    | +10 个百分点 |

反馈循环将通过率从 80% 提升至 90%。在 4 道零样本失败的题目中，反馈成功修复了 2 道（50%）。

#### 5.1.2 改善与失败案例

反馈成功修复的两道题为 Prob109\_fsm1 和 Prob127\_lemmings1，均属于”代码接近正确但存在局部状态转移错误”的场景。反馈未能修复的两道题为 Prob082\_lfsr32（LFSR 抽头多项式属于精确数学知识）和 Prob140\_fsm\_hdlc（HDLC 协议边界条件极为复杂）。

#### 5.1.3 小结

VerilogEval-Human 基线实验初步验证了反馈循环的有效性：对局部逻辑错误修复效果明显，但对领域知识缺口和深层协议逻辑改善有限。

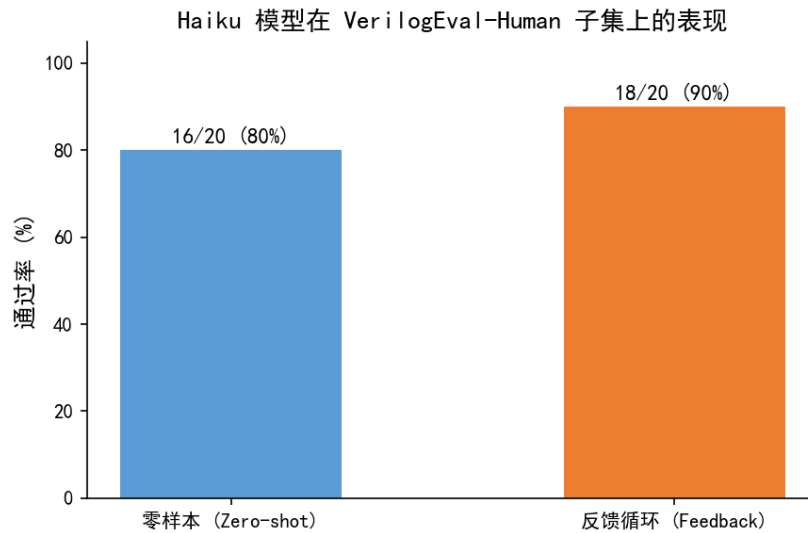


图 5.1 VerilogEval-Human 20 题零样本与反馈通过率对比

## 5.2 RTLLM\_STUDY\_12 正式实验结果

### 5.2.1 总体结果

在 RTLLM\_STUDY\_12 上使用三个模型进行 Zero-shot 与 Feedback 对比, 结果如表5.2所示。

表 5.2 RTLLM\_STUDY\_12 三模型正式实验结果

| 模型         | Zero-shot  | Feedback    | 提升    | 改善题数 |
|------------|------------|-------------|-------|------|
| Haiku 4.5  | 5/12 (42%) | 6/12 (50%)  | +8pp  | 2    |
| Sonnet 4.6 | 5/12 (42%) | 7/12 (58%)  | +17pp | 2    |
| GPT-5.4    | 6/12 (50%) | 10/12 (83%) | +33pp | 4    |

三组实验的 API 错误率均为零。

### 5.2.2 关键发现

反馈对所有模型均有正向增益, 验证了反馈循环在更难 benchmark 上的基础有效性。值得注意的是, 强模型从反馈中获益最大——GPT-5.4 的增益(+33 个百分点) 远超 Sonnet (+17 个百分点) 和 Haiku (+8 个百分点)。这表明并非弱模型最需要反馈, 而是强模型最能利用反馈。

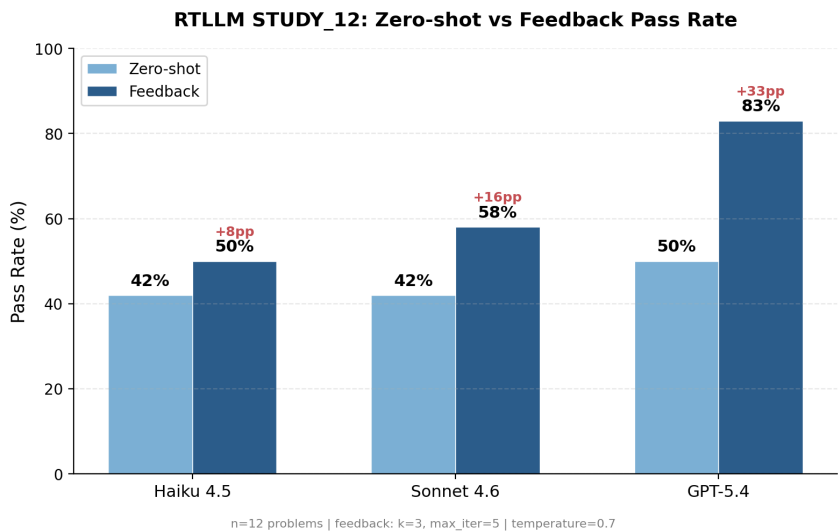


图 5.2 RTLLM\_STUDY\_12 三模型通过率对比

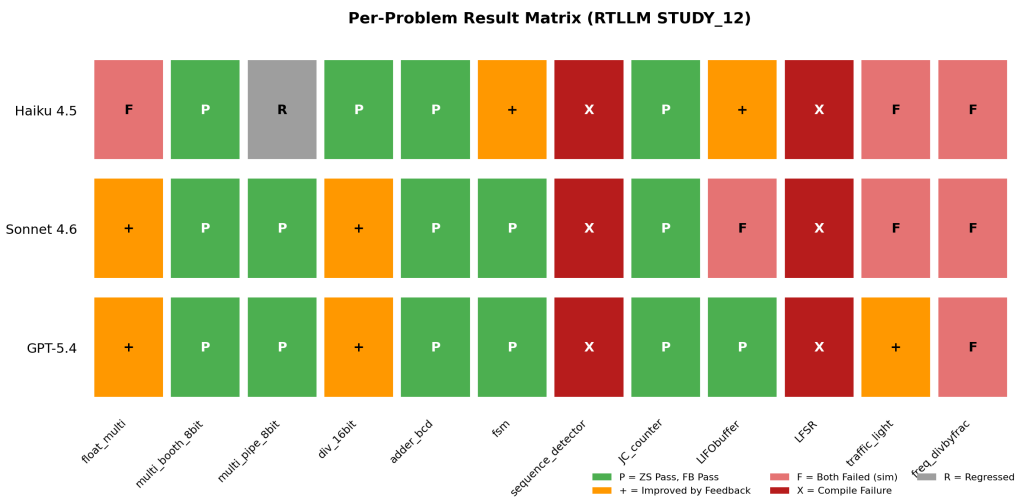


图 5.3 RTLLM\_STUDY\_12 逐题结果矩阵

实验还发现存在仅反馈可解的题目：GPT-5.4 的 LFSR 和 traffic\_light 在零样本下所有模型均失败，仅在反馈条件下通过。同时存在天花板题：sequence\_detector 和 freq\_divbyfrac 在所有模型和条件下均失败。

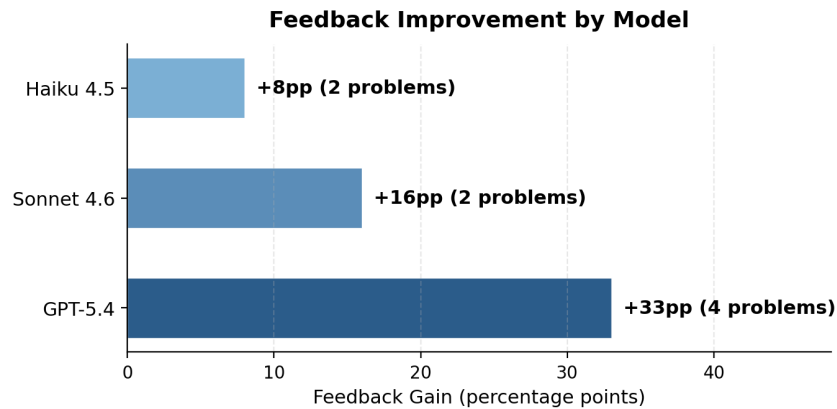


图 5.4 各模型 Feedback 增益对比

### 5.3 Feedback vs Retry-only 消融实验

#### 5.3.1 实验设计

为区分反馈循环中”多次重试”与”错误反馈信息”的各自贡献，在 GPT-5.4 和 Sonnet 4.6 上运行三条件消融实验（详见第 4 章）。

#### 5.3.2 GPT-5.4 消融结果

GPT-5.4 的单轮消融结果为：Zero-shot 50% (6/12), Retry-only 67% (8/12), Feedback 83% (10/12)。在该单轮结果中，多采样贡献 +17 个百分点，反馈信息贡献 +16 个百分点，反馈信息约占总提升的 49%。需要注意的是，稳定性实验（4 轮均值，见 5.4 节）修正后的比例为反馈信息约占 57%。

#### 5.3.3 关键证据：仅反馈可解的题目

sequence\_detector 和 traffic\_light 在 15 次独立重试中从未通过，但反馈在 2-3 轮内成功修复。这为错误反馈信息提供了独立采样无法获得的修复信号提供了直接证据。



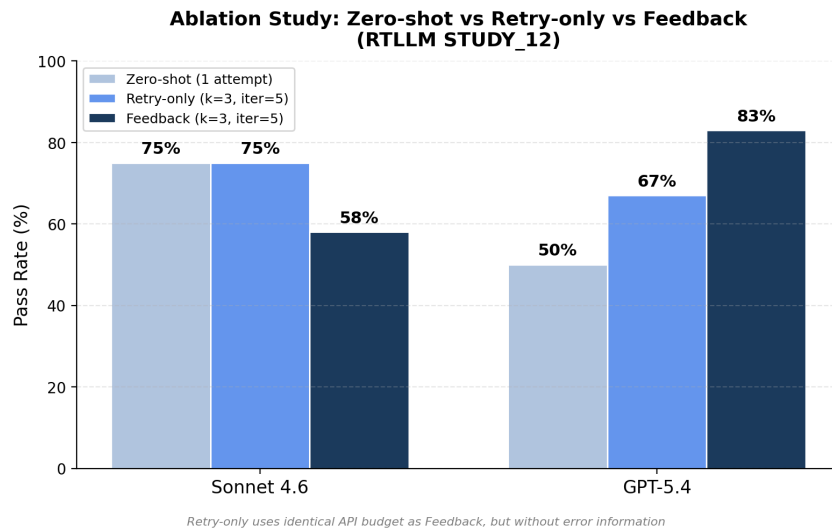
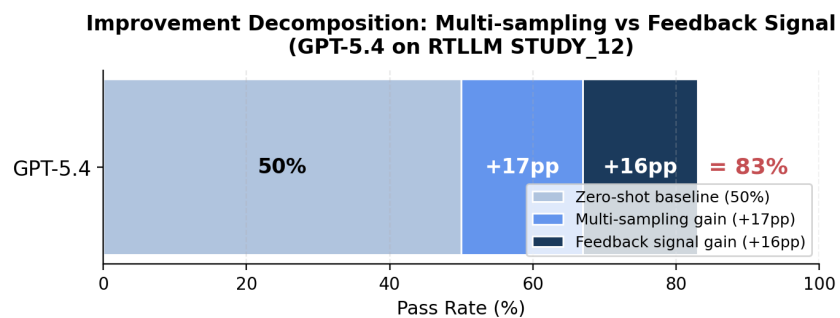


图 5.5 零样本/重试/反馈三条件对比



*sequence\_detector & traffic\_light: only solvable by feedback (15 retries all failed)*

图 5.6 GPT-5.4 反馈收益来源分解



### 5.3.4 Sonnet 消融结果

Sonnet 4.6 的单轮消融结果显示 Feedback (58%) 低于 Retry-only (75%), 但该结果受单轮随机波动影响较大。Sonnet 的消融结论需要结合稳定性实验综合判断。

## 5.4 稳定性重复实验

### 5.4.1 GPT-5.4 稳定性结果

表 5.3 GPT-5.4 稳定性实验统计

| 统计量      | Zero-shot    | Retry-only  | Feedback    |
|----------|--------------|-------------|-------------|
| 均值       | 50.0%        | 62.5%       | 79.2%       |
| 标准差      | $\pm 11.8\%$ | $\pm 4.2\%$ | $\pm 4.2\%$ |
| FB>RO 轮次 | —            | —           | 4/4         |

在 4 轮独立重复中, Feedback 始终优于 Retry-only, 且无一例外。反馈不仅提升了平均通过率, 还将标准差从 11.8% (零样本) 压缩至 4.2%。

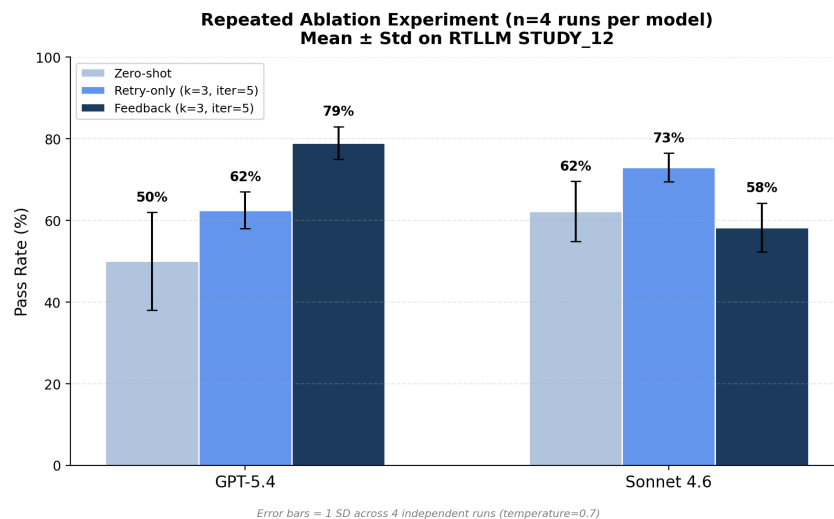


图 5.7 稳定性实验三条件均值与标准差

### 5.4.2 Sonnet 4.6 稳定性结果

Sonnet 上 Feedback 低于 Retry-only 的结论在 4 轮中稳定成立。

表 5.4 Sonnet 4.6 稳定性实验统计

| 统计量      | Zero-shot   | Retry-only  | Feedback    |
|----------|-------------|-------------|-------------|
| 均值       | 62.5%       | 72.9%       | 58.3%       |
| 标准差      | $\pm 7.2\%$ | $\pm 3.6\%$ | $\pm 5.9\%$ |
| FB>RO 轮次 | —           | —           | 0/4         |

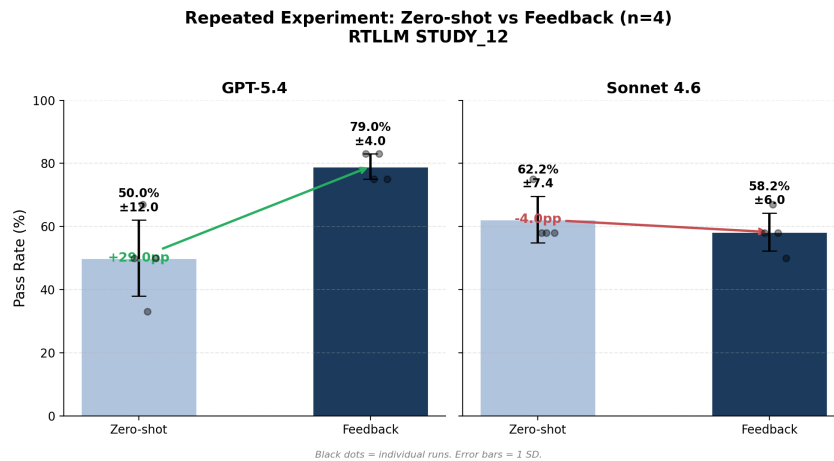


图 5.8 正式实验重复结果对比

### 5.4.3 模型依赖性分析

GPT-5.4 与 Sonnet 4.6 对反馈的响应截然相反：GPT-5.4 上 FB 相对 RO 的增益为 +16.7 个百分点，Sonnet 上为 -14.6 个百分点。这一两极分化结果在 4 轮重复中完全稳定，说明反馈机制的有效性具有模型依赖性。

## 5.5 反馈粒度实验

### 5.5.1 总体结果

### 5.5.2 倒 U 型趋势

实验结果呈现出“倒 U 型”趋势：通过率随反馈信息量的增加先升后降。L2 和 L3 达到峰值，而 L4 反而出现下降。



表 5.5 反馈粒度实验通过率

| 级别              | GPT-5.4     | Sonnet 4.6 |
|-----------------|-------------|------------|
| L0 Zero-shot    | 58% (7/12)  | 50% (6/12) |
| L1 Retry-only   | 75% (9/12)  | 67% (8/12) |
| L2 Compile-only | 92% (11/12) | 75% (9/12) |
| L3 Succinct     | 92% (11/12) | 67% (8/12) |
| L4 Rich         | 83% (10/12) | 67% (8/12) |

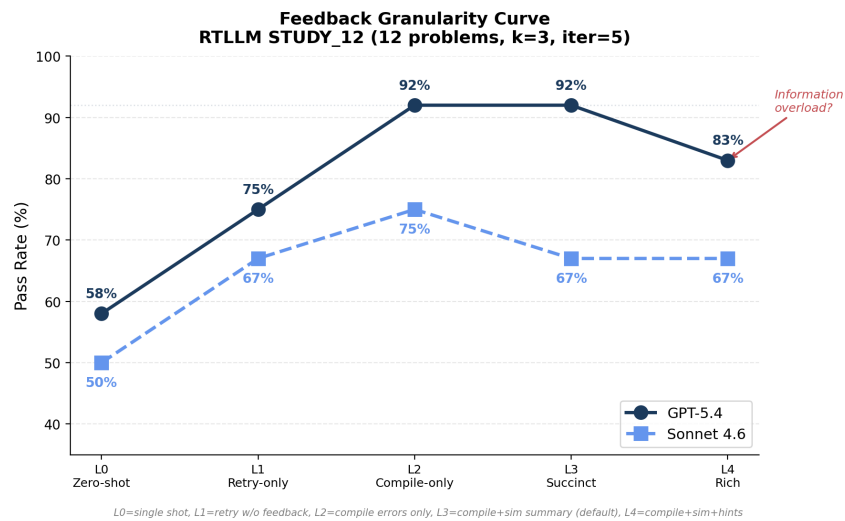


图 5.9 反馈粒度通过率曲线（倒 U 型）

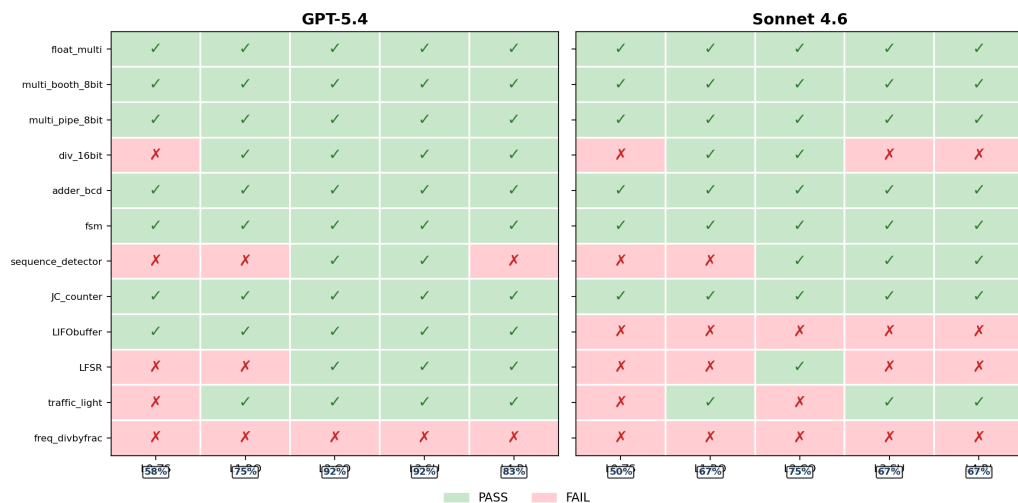
**Feedback Granularity Matrix**  
RTLLM STUDY\_12 — Per-Problem Results

图 5.10 反馈粒度逐题结果矩阵



### 5.5.3 编译反馈的高效性

L1→L2 是最大的跃升点，表明即使是最基础的编译错误反馈也具有极高的修复价值。L2 Compile-only 在两个模型上均为最优或并列最优，是最稳健的通用选择。

## 5.6 多轮对话反馈实验

### 5.6.1 实验说明

本节报告的多轮对话反馈结果均来自 v2 修正版。v1 版本因反馈数据源 bug 已在权威实验索引中标记为 Superseded。

### 5.6.2 GPT-5.4 结果

表 5.6 多轮对话反馈 v2 实验结果 (GPT-5.4)

| 条件          | 通过率         |
|-------------|-------------|
| A: ST $k=3$ | 83% (5/6)   |
| D: ST $k=1$ | 83% (5/6)   |
| B: MT $k=1$ | 100% (5/5)* |
| C: CO $k=3$ | 83% (5/6)   |

\*freq\_divbyfrac 因 API 超时排除。

### 5.6.3 Sonnet 4.6 结果

表 5.7 多轮对话反馈 v2 实验结果 (Sonnet 4.6)

| 条件          | 通过率       |
|-------------|-----------|
| A: ST $k=3$ | 17% (1/6) |
| D: ST $k=1$ | 33% (2/6) |
| B: MT $k=1$ | 50% (3/6) |
| C: CO $k=3$ | 33% (2/6) |

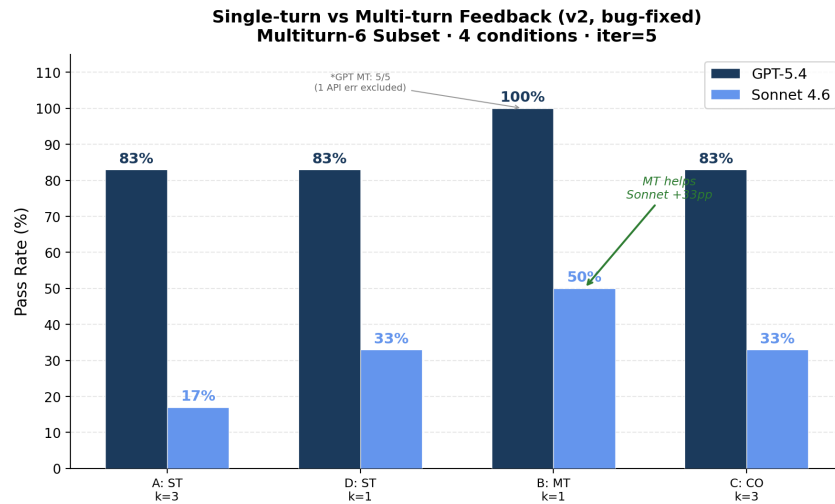


图 5.11 多轮对话反馈 v2 条件对比

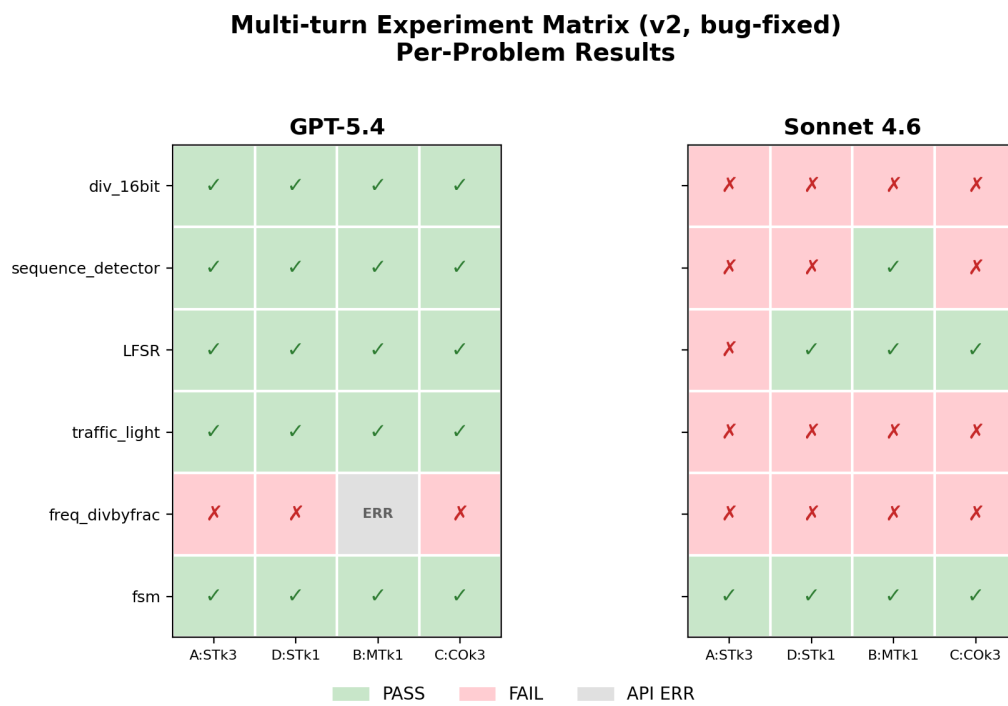


图 5.12 多轮对话反馈 v2 逐题矩阵

5.6.4 分析

多轮对话模式在两个模型上均表现出优势 (+17 个百分点), 表明持续的对话上下文有助于模型理解修复历史。但该实验在 6 题子集上进行且为单次运行, 结论需谨慎表述。

5.7 提示词策略实验

5.7.1 总体结果

使用 GPT-5.4 在 STUDY\_12 上对比四种提示策略, 结果如表5.8所示。

表 5.8 提示词策略实验结果 (GPT-5.4)

| 策略               | Zero-shot  | Feedback    | FB 增益 |
|------------------|------------|-------------|-------|
| P0: Base         | 58% (7/12) | 92% (11/12) | +34pp |
| P1: CoT          | 50% (6/12) | 92% (11/12) | +42pp |
| P2: Few-shot     | 67% (8/12) | 92% (11/12) | +25pp |
| P3: Few-shot+CoT | 58% (7/12) | 92% (11/12) | +34pp |

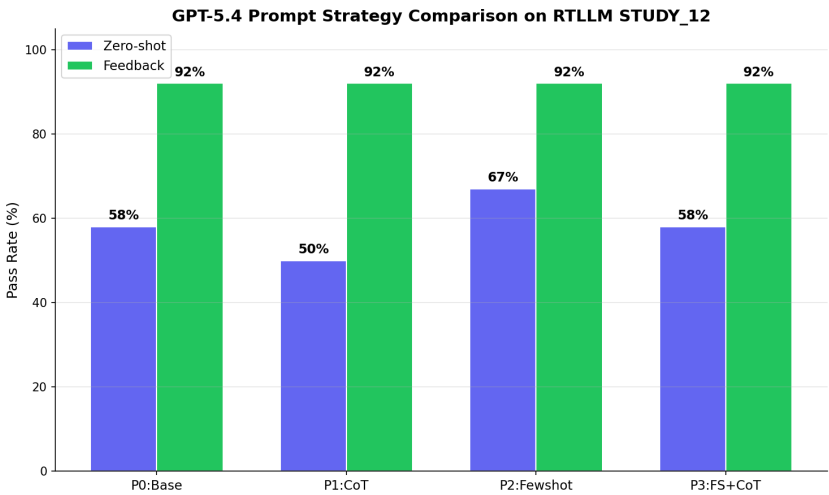


图 5.13 提示词策略零样本与反馈对比

### 5.7.2 Zero-shot 阶段的策略差异

Few-shot (P2) 是零样本最优策略 (67%), 相比 Base (P0) 提升 +9 个百分点。CoT (P1) 反而略有负面影响 (50%, -8 个百分点)。

### 5.7.3 Feedback 的”拉平效应”

四种策略在引入反馈后全部收敛至 92% (11/12), 仅 `freq_divbyfrac` 在所有策略下均未通过。这一”拉平效应”表明: 反馈循环是代码质量的主导因素, 初始提示词策略更多影响零样本起点。

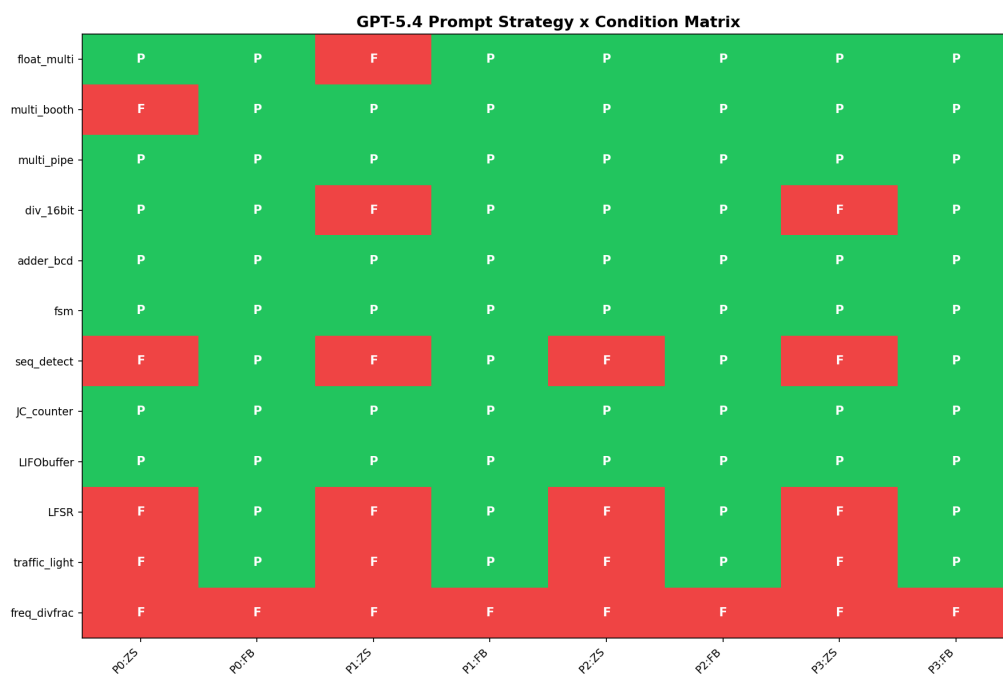


图 5.14 提示词策略逐题结果矩阵

## 5.8 典型案例分析

### 5.8.1 float\_multi: 反馈修复最难设计

`float_multi` (IEEE 754 浮点乘法器) 是 STUDY\_12 中难度最高的题目。三个模型在零样本下全部失败, 但 Sonnet 和 GPT-5.4 均在反馈的第 1 轮迭代中修复成功, 而 Haiku 即使有反馈仍无法修复。





### 5.8.2 LIFObuffer: 反馈拉起弱模型

GPT-5.4 零样本即通过, Haiku 零样本失败但反馈 1 轮修复, Sonnet 反而无法通过反馈修复。Haiku 通过反馈追平了 GPT-5.4 的零样本水平, 说明反馈机制在资源受限场景下提供了”以计算换准确度”的可行路径。

### 5.8.3 天花板题分析

sequence\_detector 在正式实验中所有模型和条件均编译失败, 暴露了当前框架的结构性局限。freq\_divbyfrac 在所有实验条件下均未通过。值得注意的是, sequence\_detector 在后续的消融稳定性实验和提示词策略实验中被成功修复, 说明该题并非绝对不可解, 而是对初始生成路径敏感。

## 5.9 综合讨论

### 5.9.1 反馈机制的有效性

在本文实验设置下, 反馈循环在 VerilogEval-Human 和 RTLLM\_STUDY\_12 两个 benchmark 上均表现出正向增益。在 RTLLM 上, GPT-5.4 的增益达到 +33 个百分点, 且该增益在 4 轮重复实验中稳定存在 ( $79.2\% \pm 4.2\%$ )。

### 5.9.2 反馈收益的来源

消融实验和稳定性实验共同揭示了反馈收益的双重来源: 约 43% 来自多次采样机会, 约 57% 来自错误反馈信息本身。后者的独立价值通过”仅反馈可解的题目”得到了证明。

### 5.9.3 模型依赖性

反馈机制的有效性存在模型依赖性。GPT-5.4 上反馈带来稳定的 +16.7 个百分点增益, 而 Sonnet 4.6 上反馈反而导致 -14.6 个百分点的回退, 且这两个结论均在 4 轮重复中完全稳定。



#### 5.9.4 反馈信息量的边界

在本文实验设置下，粒度实验揭示了一个重要的实践启示：更多的反馈信息并不总是更好。L2 Compile-only 是最稳健的通用选择。

#### 5.9.5 反馈组织方式与提示词策略

多轮对话实验（v2）表明持续对话上下文有助于修复。提示词策略实验表明反馈循环将所有策略拉平至 92%，证明反馈是主导修复因素。

#### 5.9.6 局限性

本文实验存在以下局限性：RTLLM\_STUDY\_12 仍为 12 题子集；部分实验为单次运行或小子集，统计效力有限；Sonnet 上反馈无效的原因尚未完全明确；天花板题暴露了当前框架对复杂算法生成的局限；实验仅使用 Icarus Verilog 作为仿真工具。

#### 5.10 本章小结

本章通过七组递进式实验系统验证了 AutoChip 风格反馈循环的有效性和边界条件。核心结论包括：反馈循环在两个 benchmark 上均有效，GPT-5.4 上增益最大且最稳定；反馈信息贡献约占总提升的 57%；反馈效果具有模型依赖性；反馈信息量存在倒 U 型边界；多轮对话是有潜力的改进方向；提示词策略与反馈循环为互补关系。



## 6 总结与展望

### 6.1 本文工作总结

本文围绕”基于大语言模型与 EDA 工具反馈的 RTL 代码自动生成与自修复”这一研究问题，完成了以下工作。

在系统实现方面，本文构建了一个 AutoChip 风格的 RTL 自动生成与反馈修复原型系统，实现了完整的”生成—编译—仿真—反馈—修复”闭环，包括任务加载（支持 VerilogEval 和 RTLLM 双 benchmark 适配）、大语言模型统一 API 调用、Verilog 代码提取与编译仿真执行、多粒度反馈信息构建、多种提示词策略支持，以及迭代修复循环控制。

在 benchmark 接入方面，本文在中期阶段使用 VerilogEval-Human 20 题子集完成了基础验证实验，随后引入 RTLLM 2.0 作为更具挑战性的评估基准。通过对 RTLLM 全部 50 个设计的 Icarus Verilog 兼容性审计（41/50 完全可用），从中精选 12 道高难度、多类别覆盖的设计构成 RTLLM\_STUDY\_12 正式研究子集。

在实验研究方面，本文设计并执行了七组递进式实验，覆盖了基线验证、正式实验、消融实验、稳定性重复实验、反馈粒度实验、多轮对话反馈实验和提示词策略实验。在工程质量方面，建立了完善的实验产物管理和审计机制。

### 6.2 主要实验结论

基于七组实验的结果，本文得出以下主要结论。

EDA 工具反馈能够提升 RTL 代码生成正确率。在 VerilogEval-Human 上，Haiku 模型的通过率从 80% 提升至 90%；在 RTLLM\_STUDY\_12 上，三个模型均观察到反馈带来的通过率提升。在更难 benchmark 与强模型条件下，反馈的价值更为突出：GPT-5.4 从 50% 提升至 83%，绝对提升 33 个百分点。

反馈的收益并非简单的多次重试。消融实验表明，GPT-5.4 上反馈相对于 retry-only 的增益平均为 +16.7 个百分点，且存在仅反馈可解而 15 次独立重试均无法通过的设计题目。稳定性实验确认反馈信息约贡献了总提升的 57%，多采样贡献约 43%。

反馈效果具有模型依赖性。GPT-5.4 上反馈始终优于 retry-only（4/4 轮均成立），而 Sonnet 4.6 上反馈在当前设置下反而不如 retry-only（0/4 轮优于 retry-only）。这一结论在



重复实验中完全稳定。

反馈信息量存在最佳区间。在本文实验设置下，反馈粒度实验呈现倒 U 型趋势：仅编译反馈 (L2) 和简洁反馈 (L3) 达到最优效果，而丰富反馈 (L4) 反而出现下降。多轮对话反馈和提示词策略可作为补充优化方向，其中多轮对话在控制候选数量后带来 +17 个百分点的提升，提示词策略方面反馈循环将所有策略拉平至相同水平 (92%)。

### 6.3 系统局限性

本文的系统实现和实验研究存在以下局限性。RTLLM\_STUDY\_12 包含 12 道精选题目，虽然覆盖四大类别和较高难度梯度，但仍不能代表所有类型的 RTL 设计任务。本文的正式实验集中在三个模型上，其他模型是否表现出类似的反馈响应模式尚未验证。当前系统的反馈信息主要基于仿真文本输出，缺少更精确的波形级定位。部分实验为局部探索，统计效力有限。

### 6.4 后续工作展望

#### 6.4.1 扩大 benchmark 与任务规模

可将实验从 RTLLM\_STUDY\_12 扩展到更大的任务子集或全量 RTLLM，引入其他高难度 RTL 生成 benchmark，进行更细粒度的分析。

#### 6.4.2 更多模型与自适应反馈策略

纳入更多闭源和开源大语言模型进行对照实验，探索根据模型特性和错误类型动态选择反馈策略的自适应机制。

#### 6.4.3 更精细的仿真反馈与波形分析

引入信号级差异分析，将仿真反馈从文本摘要提升到波形关键时间点定位的精度。

#### 6.4.4 检索增强与领域知识补充

针对天花板题暴露的领域知识缺口问题，探索通过检索相似的 Verilog 设计实现来补充模型的先验知识 [7]。



#### 6.4.5 形式化验证集成

探索将形式化验证工具集成到反馈循环中，利用形式化工具提供的反例作为更精确的修复线索。

#### 6.5 本章小结

本章总结了本文完成的系统实现和实验分析工作，概括了主要实验结论，诚实讨论了系统和实验的局限性，并从 benchmark 扩展、模型覆盖、反馈精细化、知识增强和形式化验证等方向展望了后续研究。



## 结论

本文构建了 AutoChip 风格的 RTL 自动生成与反馈修复原型系统，并通过七组递进式实验系统验证了 EDA 工具反馈循环的有效性和边界条件。

实验表明，反馈循环在 VerilogEval-Human 和 RTLLM\_STUDY\_12 两个 benchmark 上均能提升大语言模型生成 Verilog 代码的正确率。其中 GPT-5.4 在 RTLLM\_STUDY\_12 上的通过率从 50%（零样本）提升至 83%（反馈），增益最大且在 4 轮重复实验中稳定（ $79.2\% \pm 4.2\%$ ）。消融实验将反馈收益分解为多采样贡献（约 43%）和反馈信息贡献（约 57%），且存在仅反馈可解而多次独立重试均无法通过的设计题目。

实验同时揭示了反馈机制的边界：反馈效果具有模型依赖性；反馈信息量存在倒 U 型边界，编译反馈是最稳健的选择；天花板题暴露了当前框架对复杂算法生成的局限。多轮对话反馈和提示词策略可作为补充优化方向。

本文的工作为理解 EDA 工具反馈在大语言模型 RTL 自动生成中的作用提供了实证分析和工程参考。



## 致谢

本论文是在王锐老师的悉心指导下完成的。从毕业设计选题、研究方向确定到实验设计和论文写作，王老师始终给予了专业而耐心的指导。特别是在中期答辩时，王老师提出的“引入更强模型和更难 benchmark 来验证反馈机制有效性”的建议，为本文后中期的实验方向指明了路径，直接推动了 RTLLM 基准的引入和多模型交叉验证实验的开展。在此，谨向王锐老师致以最诚挚的感谢。

感谢北京航空航天大学计算机学院提供的本科毕业设计训练平台和学习环境，使我有机会在本科阶段接触到大语言模型与硬件设计交叉领域的前沿研究问题，并完成一项相对完整的实验性研究。

本文的研究工作得益于多个优秀的开源项目和学术资源。感谢 Icarus Verilog 项目提供的开源编译与仿真工具，使本文的自动化验证流程成为可能；感谢 NVlabs 的 VerilogEval 和香港科技大学的 RTLLM 项目提供了高质量的 RTL 生成评估基准；感谢 Python 生态和 GitHub 平台为项目开发和版本管理提供了便利。

在论文写作和代码调试过程中，合理使用了智能辅助工具进行资料整理、代码检查和表达润色，所有实验设计、实验执行和结论分析均由本人独立完成并经过严格的数据审计。

感谢在大学四年学习生活中给予我帮助和支持的同学和朋友们，与大家的交流和讨论让我受益良多。最后，感谢我的家人在求学过程中给予的理解、支持和鼓励，使我能够专注于学业和研究。



## 参考文献

- [1] CHANG K, WANG Y, REN H, et al. ChipGPT: How far are we from natural language hardware design[C]//arXiv preprint arXiv:2305.14019. [S.l.: s.n.], 2023.
- [2] THAKUR S, AHMAD B, et al. Verigen: A large language model for Verilog code generation[C]//Proceedings of the Design Automation and Test in Europe Conference (DATE). [S.l.: s.n.], 2023.
- [3] LIU M, PINCKNEY N, KHAILANY B, et al. VerilogEval: Evaluating large language models for Verilog code generation[C]//Proceedings of IEEE/ACM International Conference on Computer-Aided Design (ICCAD). [S.l.: s.n.], 2023.
- [4] LU Y, LIU S, ZHANG Q, et al. RTLLM: An open-source benchmark for design RTL generation with large language model[C]//Proceedings of the Asia and South Pacific Design Automation Conference (ASP-DAC). [S.l.: s.n.], 2024.
- [5] THAKUR S, AHMAD B, FAN Z, et al. AutoChip: Automating HDL generation using LLM feedback[C]//Proceedings of the ML for Systems Workshop at NeurIPS. [S.l.: s.n.], 2023.
- [6] TSAI Y D, LIU M, REN H. RTLFixer: Automatically fixing RTL syntax errors with large language models[J]. arXiv preprint arXiv:2311.16543, 2024.
- [7] YAO X, et al. HDLdebugger: Streamlining HDL debugging with large language models [J]. arXiv preprint arXiv:2403.11671, 2025.
- [8] BROWN T B, MANN B, RYDER N, et al. Language models are few-shot learners[C]//Advances in Neural Information Processing Systems (NeurIPS). [S.l.: s.n.], 2020.
- [9] WILLIAMS S. Icarus Verilog[EB/OL]. 2024. <https://github.com/steveicarus/iverilog>.
- [10] HDLBits. HDLBits — Verilog practice[EB/OL]. 2024. <https://hdlbits.01xz.net/>.
- [11] LIU S, FANG W, LU Y, et al. RTLCoder: Fully open-source and efficient LLM-assisted RTL code generation technique[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2024.
- [12] WEI J, WANG X, SCHUURMANS D, et al. Chain-of-thought prompting elicits reasoning in large language models[C]//Advances in Neural Information Processing Systems





- (NeurIPS). [S.l.: s.n.], 2022.
- [13] HO C T, KUO C L, KHAILANY B, et al. VerilogCoder: Autonomous Verilog coding agents with graph-based planning and abstract syntax tree (AST) tools[J]. arXiv preprint arXiv:2408.08927, 2024.
- [14] BLOCKLOVE J, GARG S, KARRI R, et al. Chip-chat: Challenges and opportunities in conversational hardware design[C]//Proceedings of the ML for Systems Workshop at NeurIPS. [S.l.: s.n.], 2023.
- [15] MENZA M U I, HASSOUN S. AIvriL: Ai-driven RTL generation with verification-in-the-loop[J]. arXiv preprint arXiv:2409.11411, 2024.
- [16] FAN R, CHEN Y, ZHAO J, et al. AutoBench: Automatic testbench generation and evaluation using LLMs for HDL design[C]//Proceedings of the ML for Systems Workshop at NeurIPS. [S.l.: s.n.], 2024.
- [17] PINCKNEY N, LIU M, KHAILANY B, et al. Revisiting VerilogEval: Newer LLMs, in-context learning, and specification-to-RTL tasks[C]//arXiv preprint arXiv:2408.11053. [S.l.: s.n.], 2024.



## 附录 A 附录 A：实验配置与运行命令

本附录记录了本文实验使用的典型运行命令模板和关键配置。

### A.1 正式实验运行命令

```
# Zero-shot
python scripts/run_rtllm_experiment.py \
  --model gpt-5.4 --mode zero-shot \
  --problems STUDY_12
```

```
# Feedback
python scripts/run_rtllm_experiment.py \
  --model gpt-5.4 --mode feedback \
  --k 3 --max-iterations 5 \
  --feedback-level succinct \
  --problems STUDY_12
```

### A.2 环境配置

实验环境：Python 3.11，Icarus Verilog 12.0，Ubuntu 22.04。API 通过第三方统一网关接入。

### A.3 核心代码模块索引



表 A.1 系统核心代码模块索引

| 模块          | 文件路径                     |
|-------------|--------------------------|
| 反馈循环控制器     | src/loop_runner.py       |
| 提示词构建器      | src/prompt_builder.py    |
| Verilog 执行器 | src/verilog_executor.py  |
| 评分器         | src/ranker.py            |
| RTLLM 加载器   | src/rtllm_loader.py      |
| 代码审计        | scripts/audit_project.py |
| 数据审计        | scripts/audit_data.py    |